

COMP3026E Distributed Systems

13. Spanner

James Jianqiao Yu

Harbin Institute of Technology, Shenzhen

Recap: Distributed Storage Systems

- Concurrency control
 - Order transactions across shards
- State machine replication
 - Replicas of a shard apply transactions in the same order decided by concurrency control

Google's Setting



- Dozens of datacenters (zones)
- Per zone, 100-1000s of servers
- Per server, 100-1000 shards (tablets)
- Every shard replicated for fault-tolerance (e.g., 5x)

Why Google Built Spanner



- 2005 – BigTable [OSDI 2006]
 - Eventually consistent across datacenters
 - Lesson: “don’t need distributed transactions”

Why Google Built Spanner

- 2005 – BigTable [OSDI 2006]
 - Eventually consistent across datacenters
 - Lesson: “don’t need distributed transactions”
- 2008? – MegaStore [CIDR 2011]
 - Strongly consistent across datacenters
 - Option for distributed transactions
 - But performance was not great...

Why Google Built Spanner

- 2005 – BigTable [OSDI 2006]
 - Eventually consistent across datacenters
 - Lesson: “don’t need distributed transactions”
- 2008? – MegaStore [CIDR 2011]
 - Strongly consistent across datacenters
 - Option for distributed transactions
 - But performance was not great...
- 2011 – Spanner [OSDI 2012]
 - Strictly Serializable Distributed Transactions
 - “We wanted to make it easy for developers to build their applications”



Motivation: Performance-Consistency Tradeoff



- Strict serializability
 - Serializability + linearizability
 - As if coding on a single-threaded, transactionally isolated machine
 - Spanner calls it external consistency
- Strict serializability makes building correct application easier
- Strict serializability is expensive
 - Performance penalty in concurrency control + Repl.
 - OCC/2PL: multiple round trips, locking, etc.

Motivation: Read-Only Transactions

- Transactions that only read data
 - Predeclared, i.e., developer uses READ_ONLY flag / interface
- Reads dominate real-world workloads
 - FB's TAO had **500 reads** vs. 1 write [ATC 2013]
 - Google Ads (F1) on Spanner from 1 DC in 24h:
 - 31.2 M single-shard read-write transactions
 - 32.1 M multi-shard read-write transactions
 - 21.5 B read-only (340 times more)
- Determines system overall performance



Before We Get to Spanner



Question Can we design a **strictly serializable**, geo-replicated, sharded system with **very fast (efficient)** read-only transactions?

- How would you design strictly serializable read-only transactions?
- OCC or 2PL: Multiple round trips and locking
- Can always read in local datacenters like COPS?
 - Maybe involved in Paxos agreement
 - Or must contact the leader
- Performance penalties
 - Round trips increase latency, especially in wide area
 - Distributed lock management is costly, e.g., deadlocks



Goal is to ...



- Make read-only transactions efficient
 - One round trip (as could be wide-area)
 - Lock-free
 - No deadlocks
 - Processing reads do not block writes, e.g., long-lived reads
 - Always succeed (do not abort)
- And strictly serializable

Leveraging the Notion of Time

- Strict serializability: a matter of real-time ordering
 - If transaction T2 starts after T1 finishes, then T2 must be ordered after T1
 - If T2 is read-only transaction, then T2 should see effects of all writes finished before T2 started

Leveraging the Notion of Time



- A similar scenario at a restaurant
 - Alice arrives, writes her name and time she arrives (e.g., 5pm) on waiting list
 - Bob then arrives, writes his name and the time (e.g., 5:10PM)
 - Then Bob is ordered after Alice on the waiting list
 - I arrive later at 5:15PM and check how many people are ahead of me by checking the waiting list by time

Leveraging the Notion of Time

- Task 1: when committing a write, tag it with the current physical time
- Task 2: when reading the system, check which writes were committed before the time this read started
- How about the serializable requirement?
 - Physical time naturally gives a total order



Invariant If T2 starts after T1 commits (finishes), then T2 must have a larger timestamp than T1

- Trivially provided by perfect clocks
- Clocks are not perfect
 - Clock skew: some clocks are faster/slower
 - Clock skew may not be bounded
 - Clock skew may not be known a priori
- T2 may be tagged a smaller timestamp than T1 due to slower clock
- Seems **impossible to have perfect clocks** in distributed systems. What can we do?

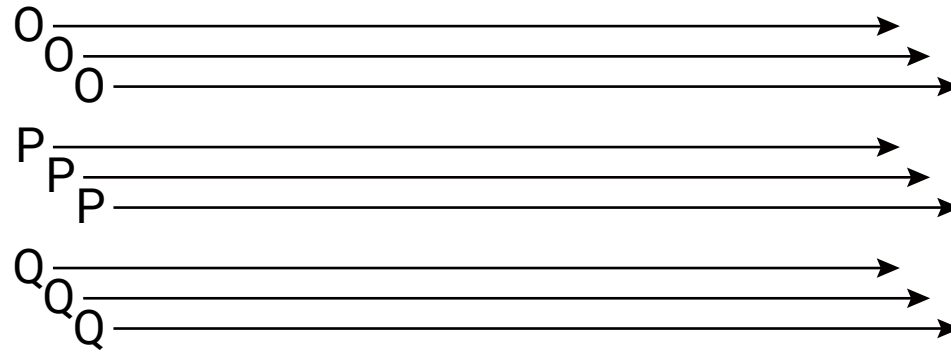
Nearly Perfect Clocks

- Partially synchronized
 - Clock skew is bounded and **known a priori**
 - My clock shows 1:30PM, then I know the absolute (real) time is in the range of 1:30 PM +/- X.
 - e.g., between 1:20PM and 1:40PM if X = 10 mins
- Clock skew is short (e.g., X = a few milliseconds)
- Enable something special, e.g., Spanner!

Spanner A globally distributed database that provides strictly serializable transactions with efficient read-only transactions

- Spanner is a **distributed database** built by Google
- Provides **strictly serializable** transactions across datacenters
- Read-only transactions are efficient (one round trip, lock-free, always succeed)
- Leverages the notion of time and partially synchronized clocks

Scale-out vs. Fault Tolerance



- Every shard replicated via MultiPaxos (akin to RAFT)
- So every “operation” within transactions across tablets actually a replicated operation within Paxos RSM
- Paxos groups can stretch across datacenters!



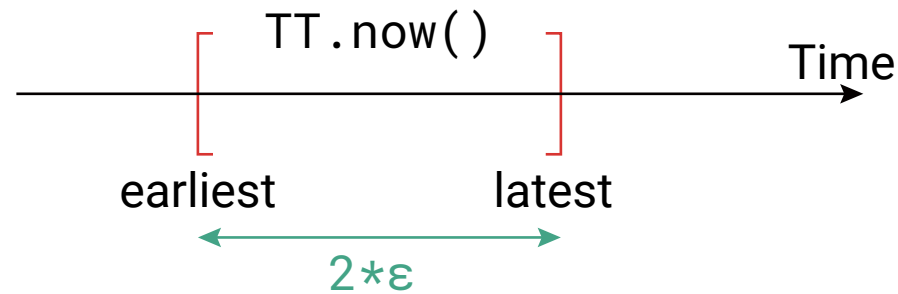
Strictly Serializable Multi-shard Transactions



- How are clocks made “nearly perfect”?
- How does Spanner leverage these clocks?
 - How are writes done and tagged?
 - How read-only transactions are made efficient?

TrueTime (TT)

- “Global wall-clock time” with bounded uncertainty
 - ϵ is worst-case clock divergence
 - Spanner’s notion of time becomes intervals, not single values
 - ϵ is 4ms on average, 2ϵ is about 10ms



TrueTime (TT)

- API (software interface)
 - `TT.now() = [earliest, latest]` # $\text{latest} - \text{earliest} = 2 * \epsilon$
 - `TT.after(t) = true` if t has passed
 - `TT.now().earliest > t` (because $t_{\text{abs}} \geq \text{TT.now().earliest}$)
 - `TT.before(t) = true` if t has not arrived
 - `TT.now().latest < t` (because $t_{\text{abs}} \leq \text{TT.now().latest}$)

TrueTime (TT)

- Implementation
 - Relies on specialized hardware, e.g., satellite and atomic clocks

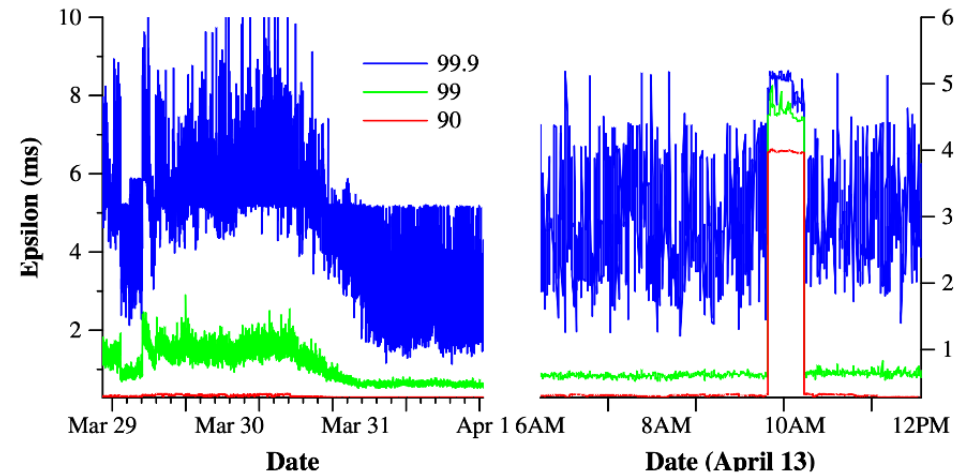
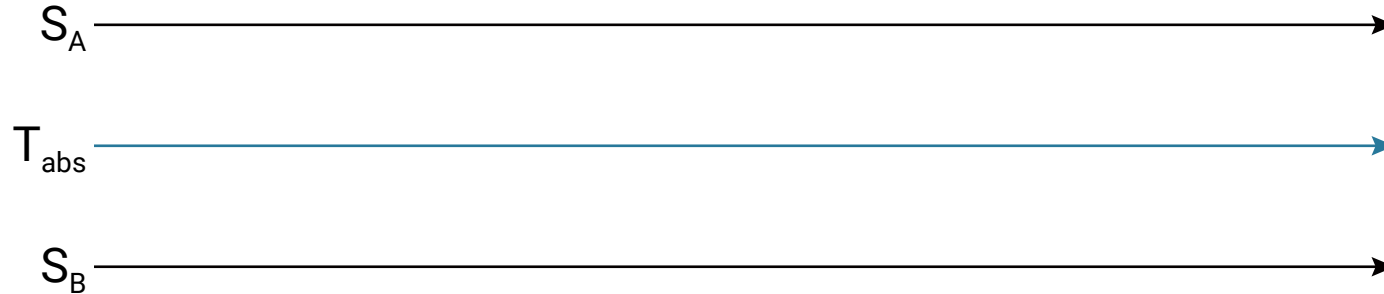


Figure 6: Distribution of TrueTime ϵ values, sampled right after timeslave daemon polls the time masters. 90th, 99th, and 99.9th percentiles are graphed.

Enforcing the Invariant

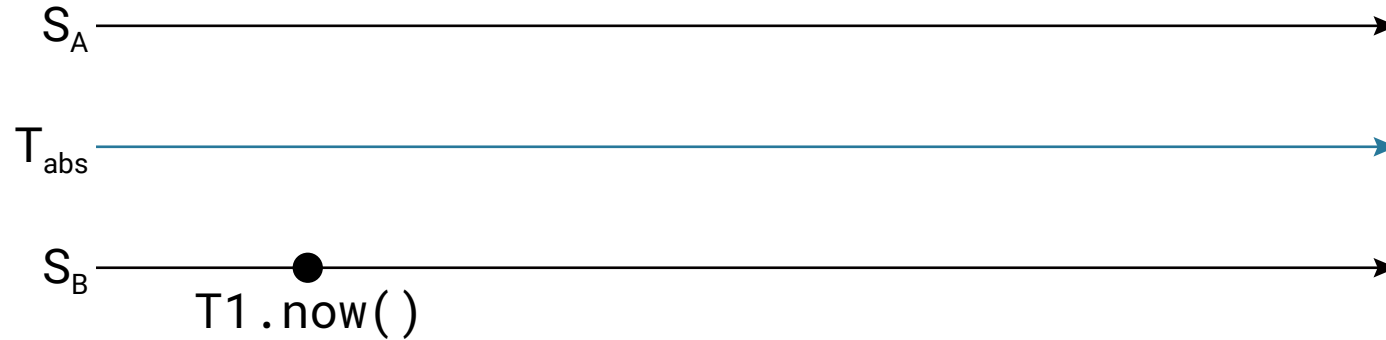
- If T2 starts after T1 commits, then T2 must have larger timestamp
- Let T1 write S_B and T2 write S_A



Perfect Clocks

Enforcing the Invariant

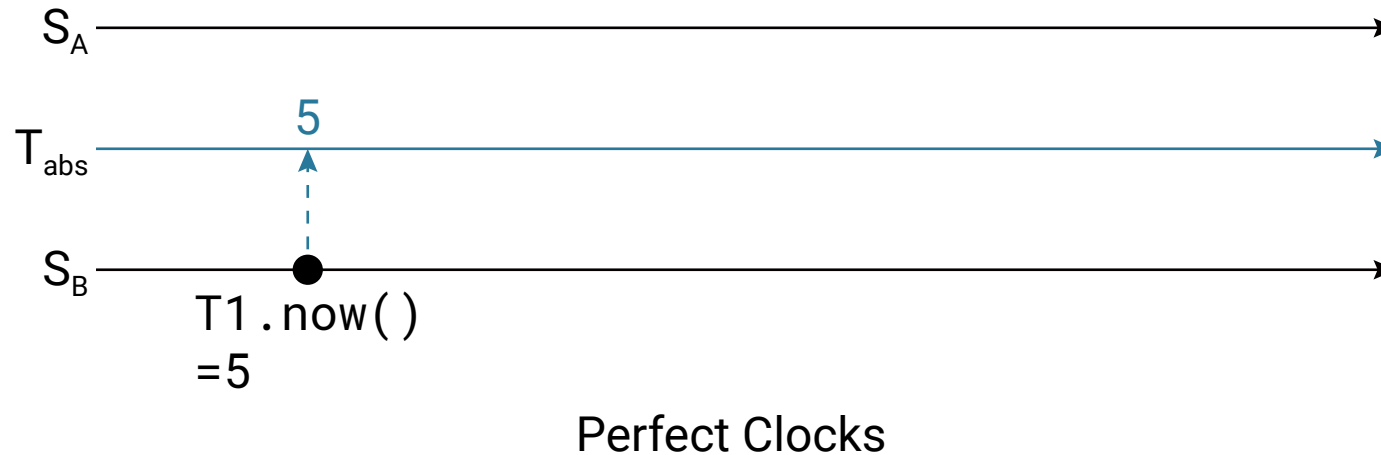
- If T2 starts after T1 commits, then T2 must have larger timestamp
- Let T1 write S_B and T2 write S_A



Perfect Clocks

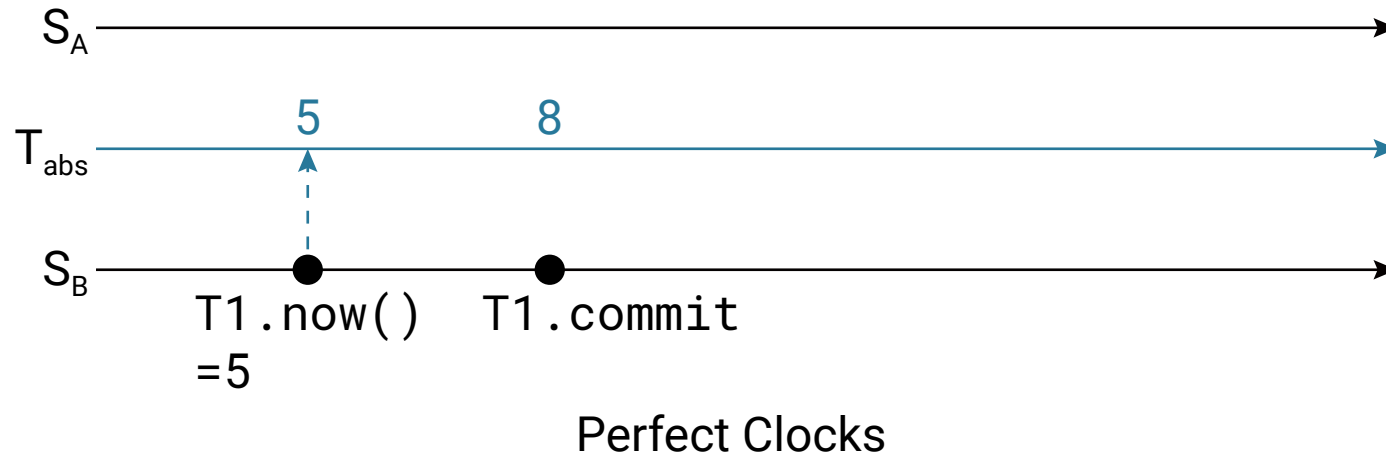
Enforcing the Invariant

- If T2 starts after T1 commits, then T2 must have larger timestamp
- Let T1 write S_B and T2 write S_A



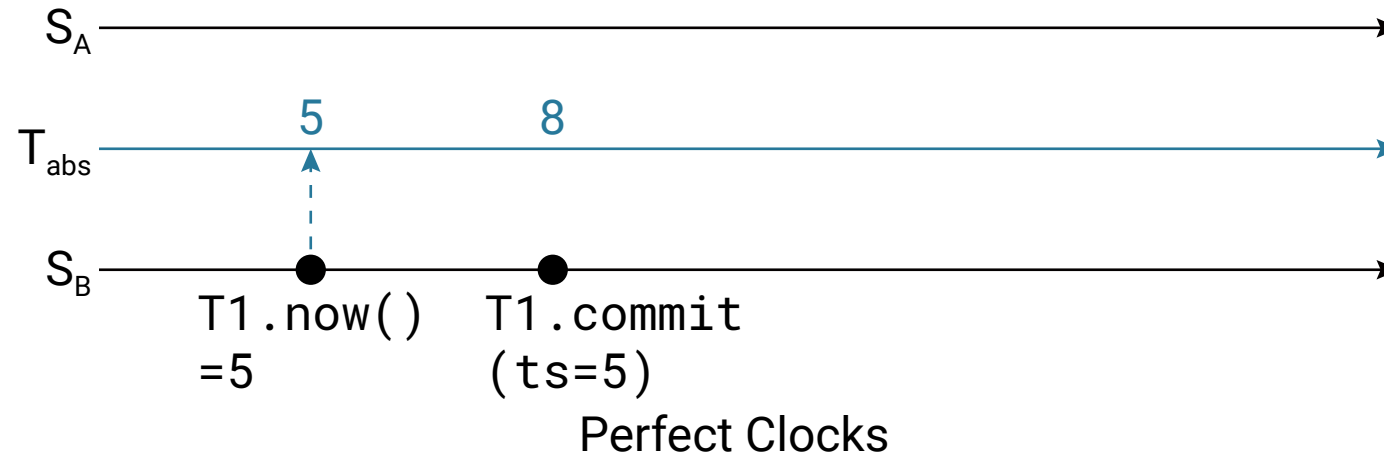
Enforcing the Invariant

- If T2 starts after T1 commits, then T2 must have larger timestamp
- Let T1 write S_B and T2 write S_A



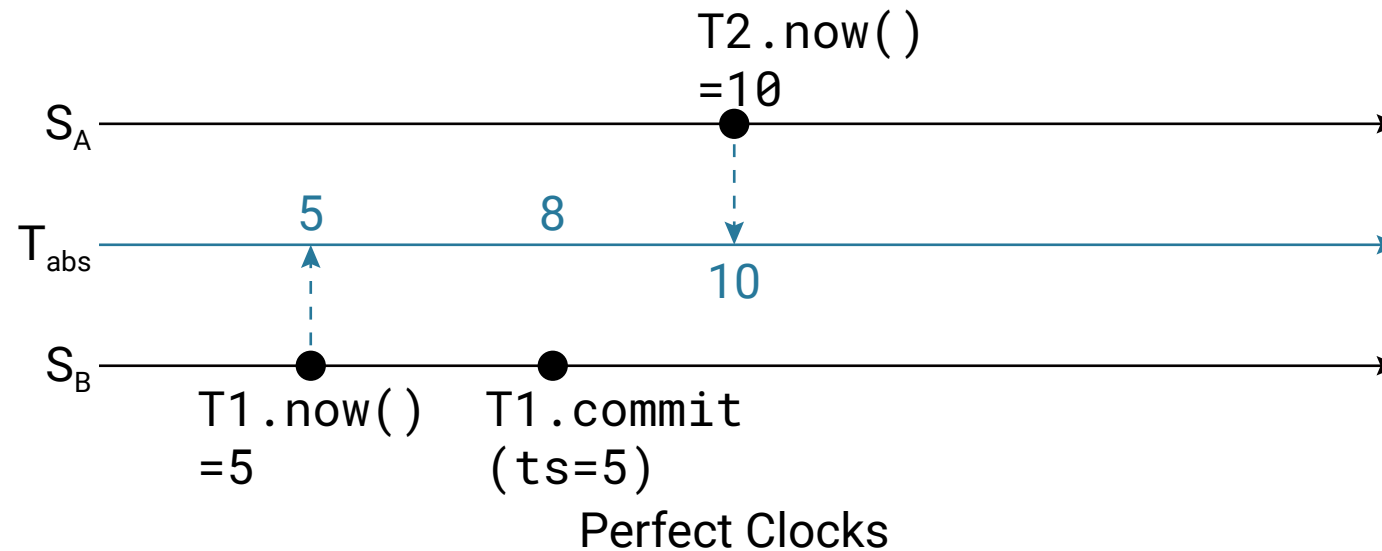
Enforcing the Invariant

- If T2 starts after T1 commits, then T2 must have larger timestamp
- Let T1 write S_B and T2 write S_A



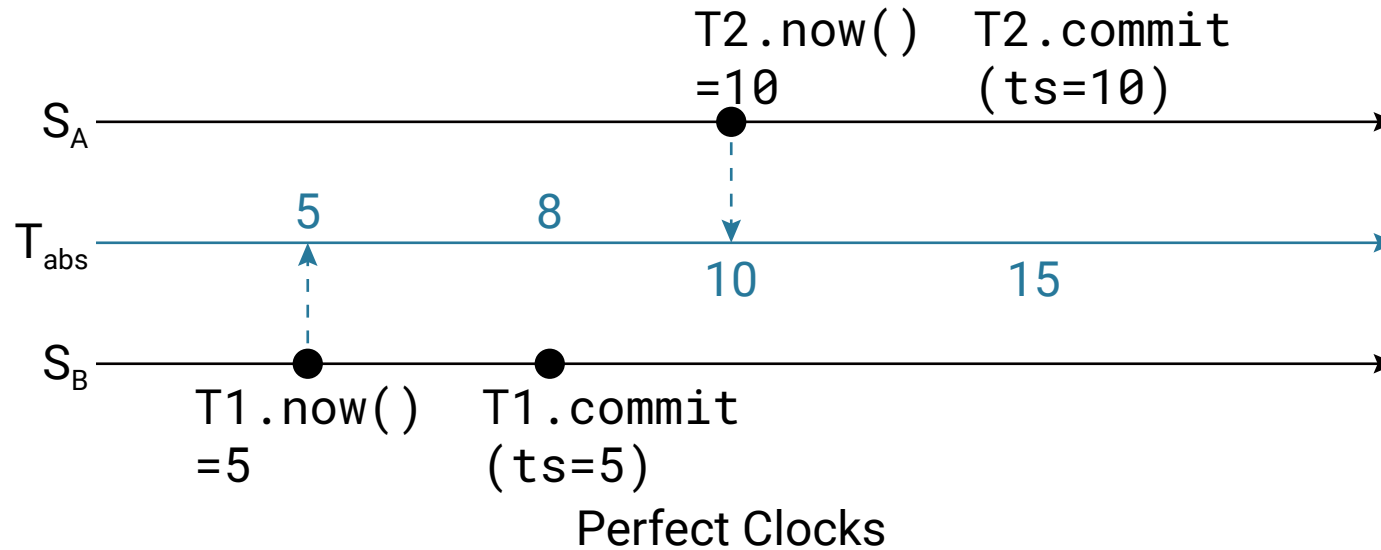
Enforcing the Invariant

- If T2 starts after T1 commits, then T2 must have larger timestamp
- Let T1 write S_B and T2 write S_A



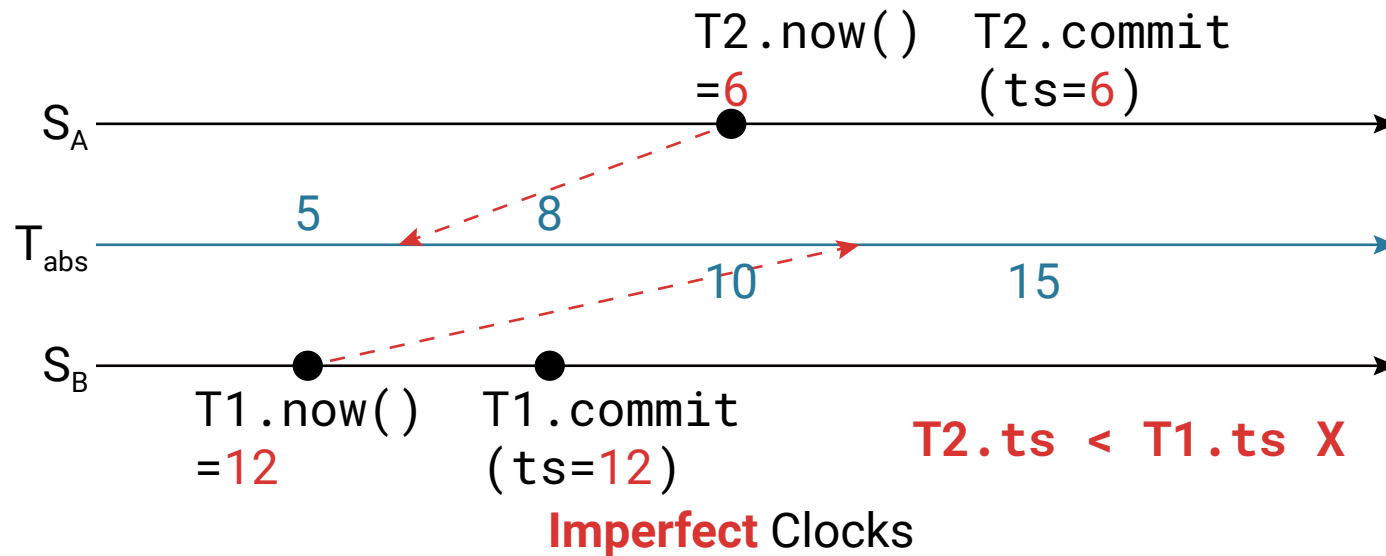
Enforcing the Invariant

- If T2 starts after T1 commits, then T2 must have larger timestamp
- Let T1 write S_B and T2 write S_A



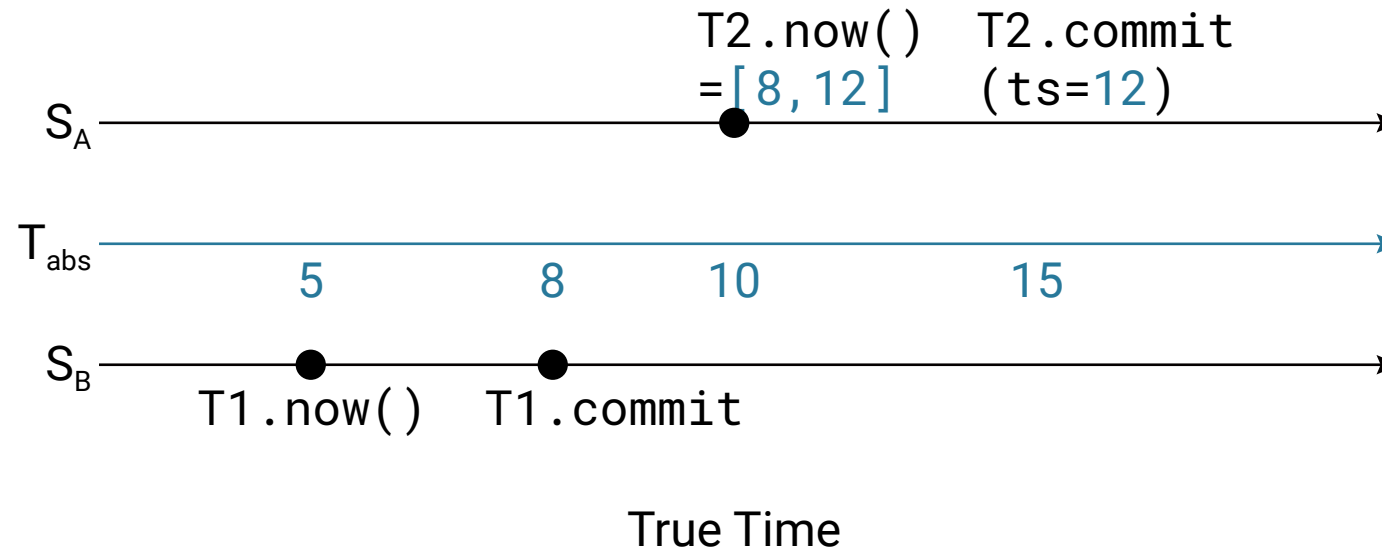
Enforcing the Invariant

- If T2 starts after T1 commits, then T2 must have larger timestamp
- Let T1 write S_B and T2 write S_A



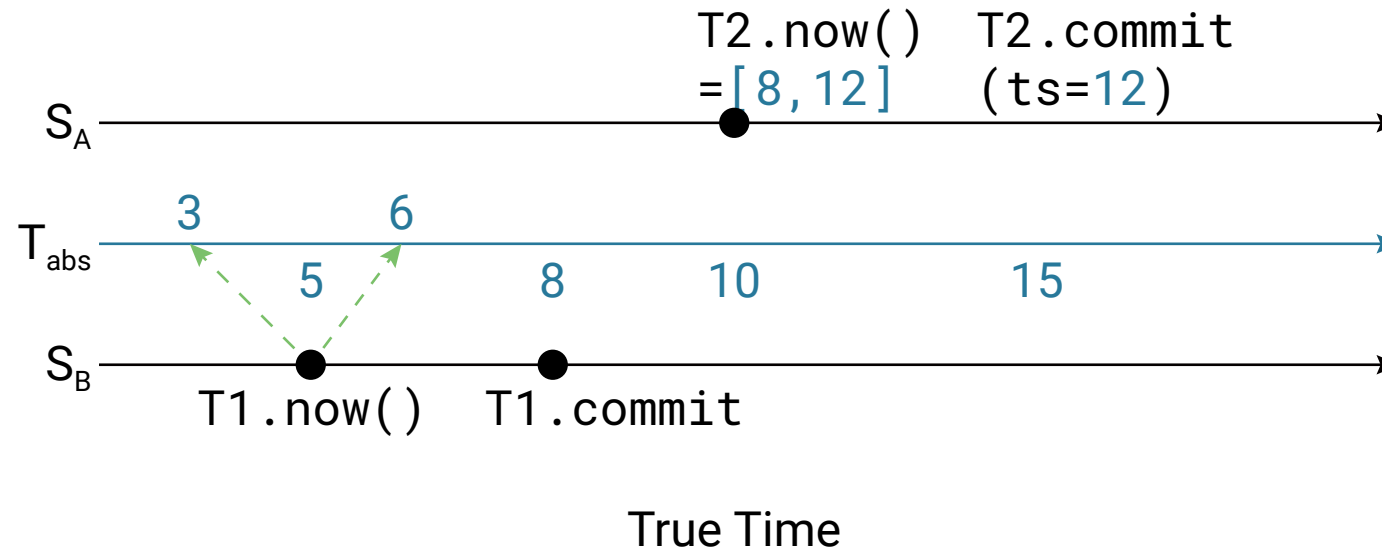
Enforcing the Invariant

- If T2 starts after T1 commits, then T2 must have larger timestamp
- Let T1 write S_B and T2 write S_A



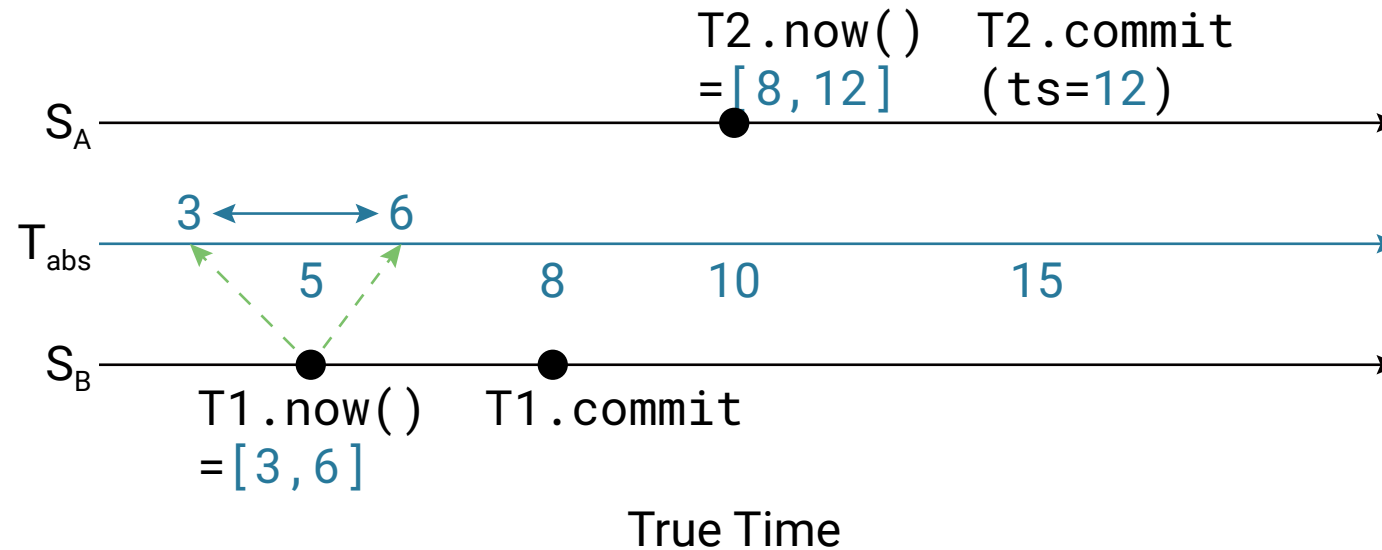
Enforcing the Invariant

- If T2 starts after T1 commits, then T2 must have larger timestamp
- Let T1 write S_B and T2 write S_A



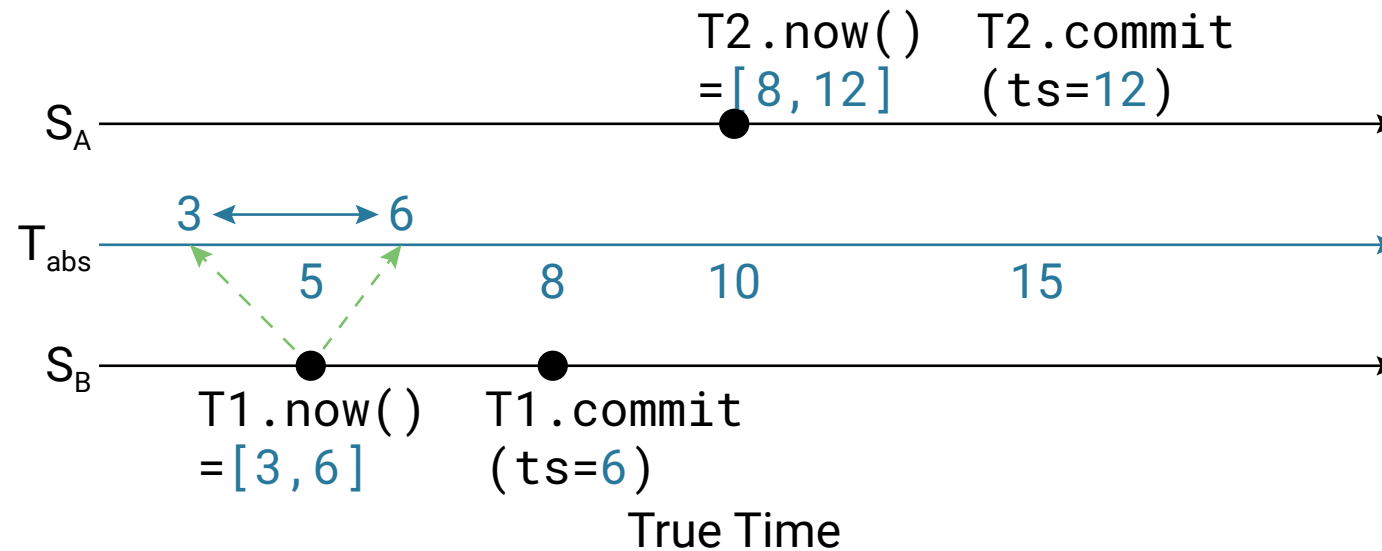
Enforcing the Invariant

- If T2 starts after T1 commits, then T2 must have larger timestamp
- Let T1 write S_B and T2 write S_A



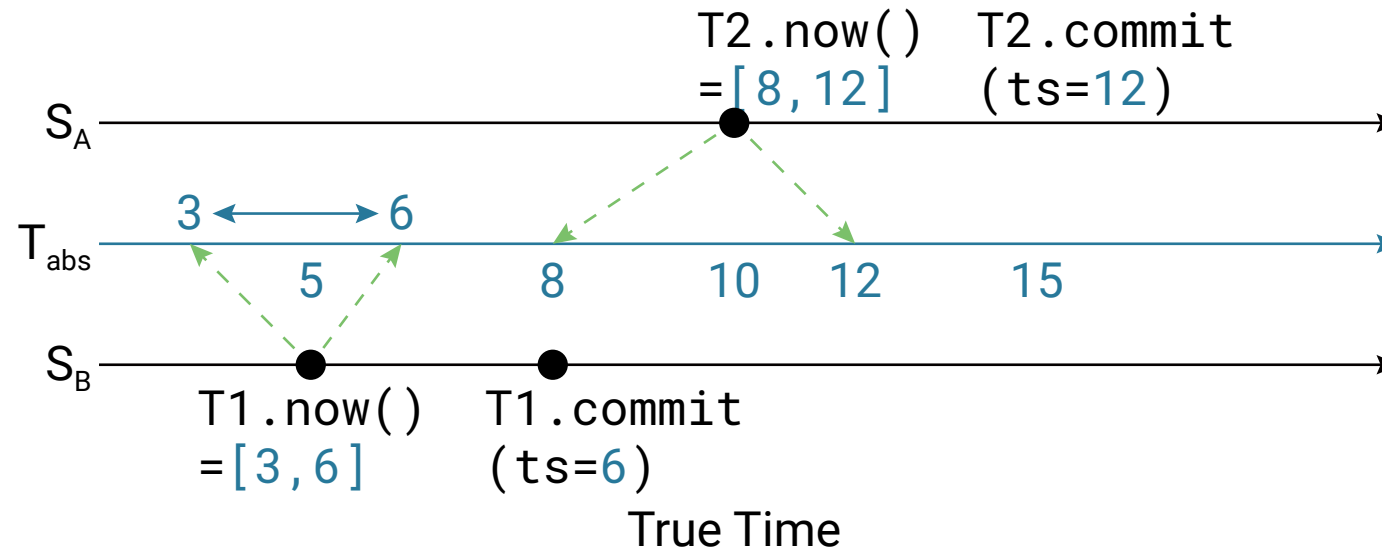
Enforcing the Invariant

- If T2 starts after T1 commits, then T2 must have larger timestamp
- Let T1 write S_B and T2 write S_A



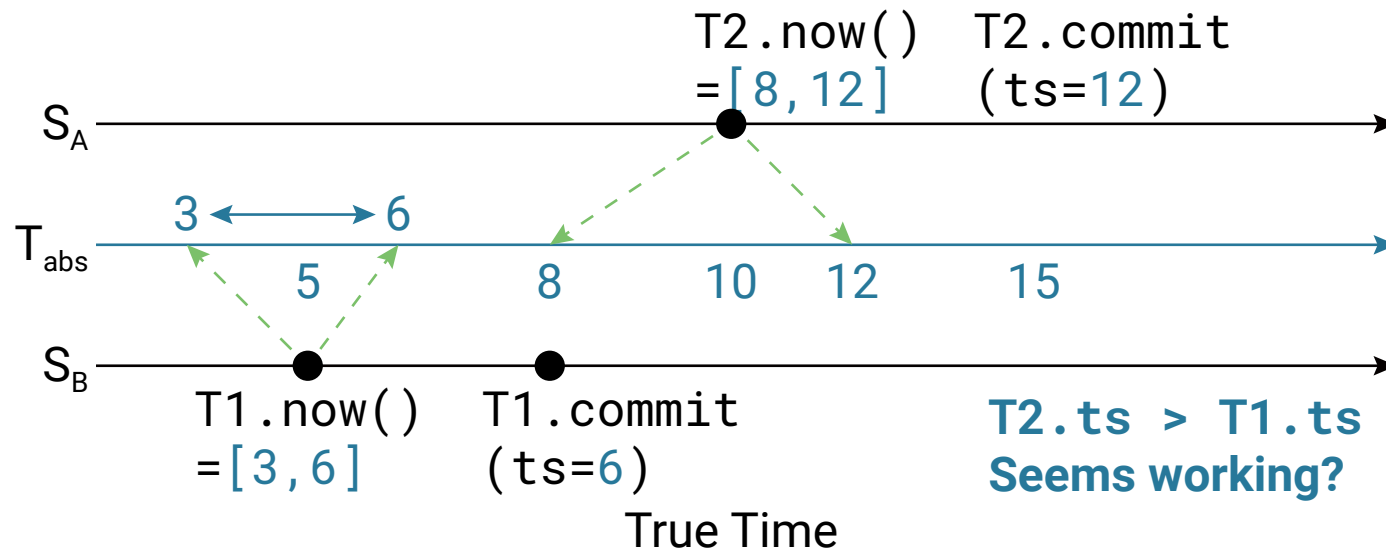
Enforcing the Invariant

- If T2 starts after T1 commits, then T2 must have larger timestamp
- Let T1 write S_B and T2 write S_A



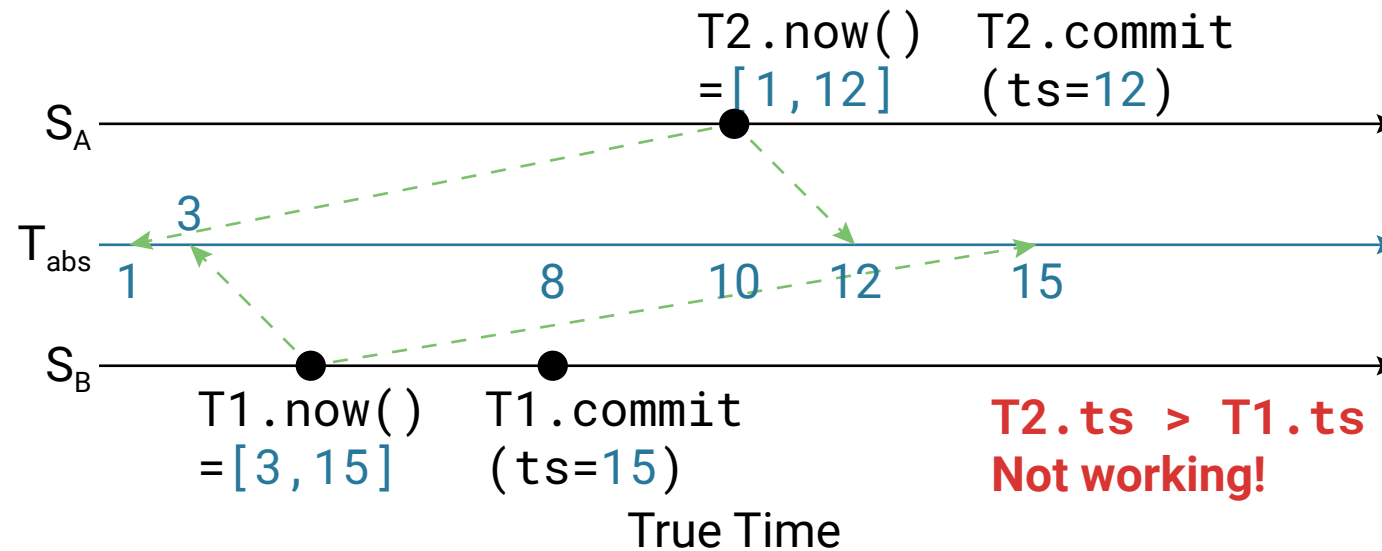
Enforcing the Invariant

- If T2 starts after T1 commits, then T2 must have larger timestamp
- Let T1 write S_B and T2 write S_A

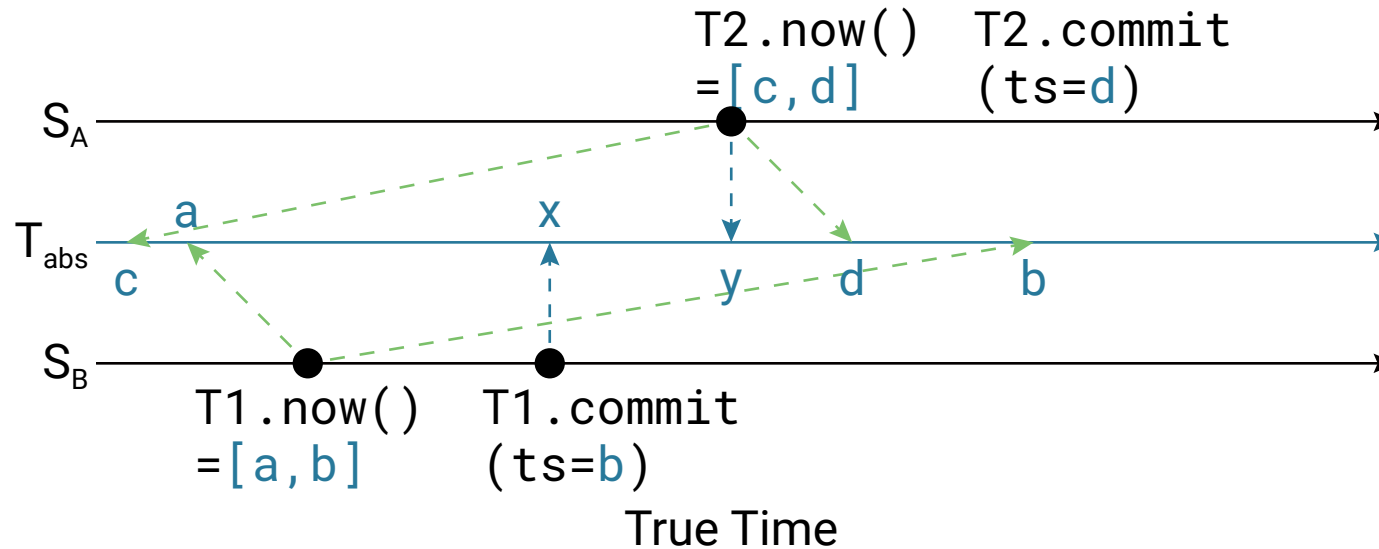


Enforcing the Invariant

- If T2 starts after T1 commits, then T2 must have larger timestamp
- Let T1 write S_B and T2 write S_A

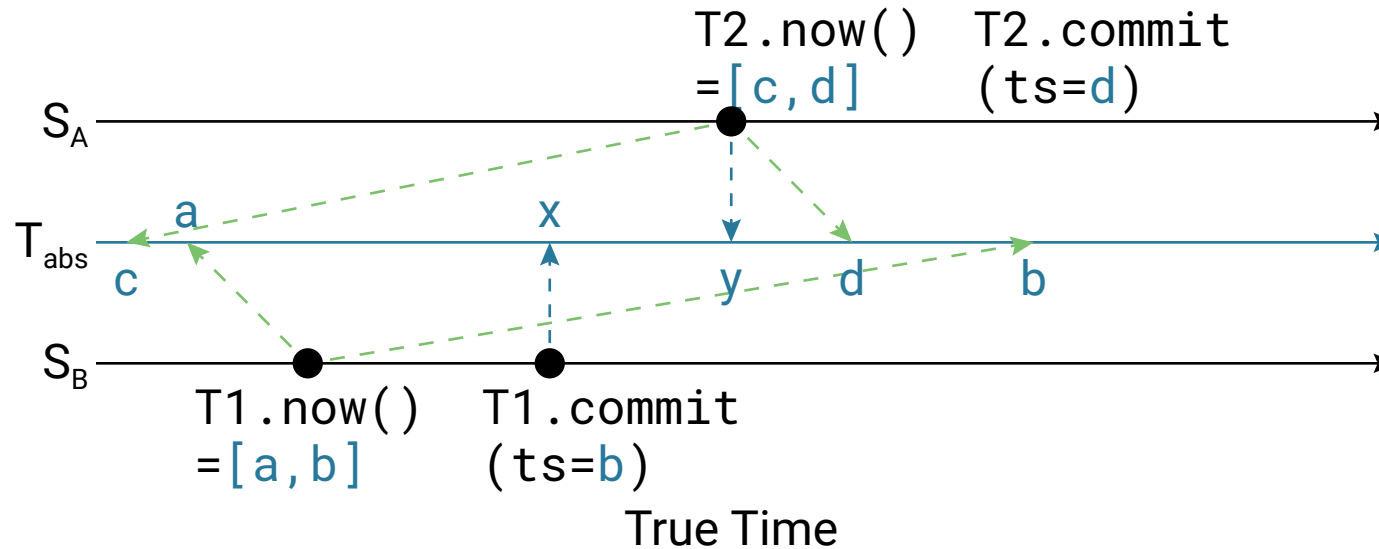


A Brain Teaser Puzzle



- We know:
 1. $x < y$, because T2 in real-time after T1 (the assumption)
 2. $c \leq y \leq d$, because TrueTime
 3. $T1.ts = b, T2.ts = d$, because how ts is assigned

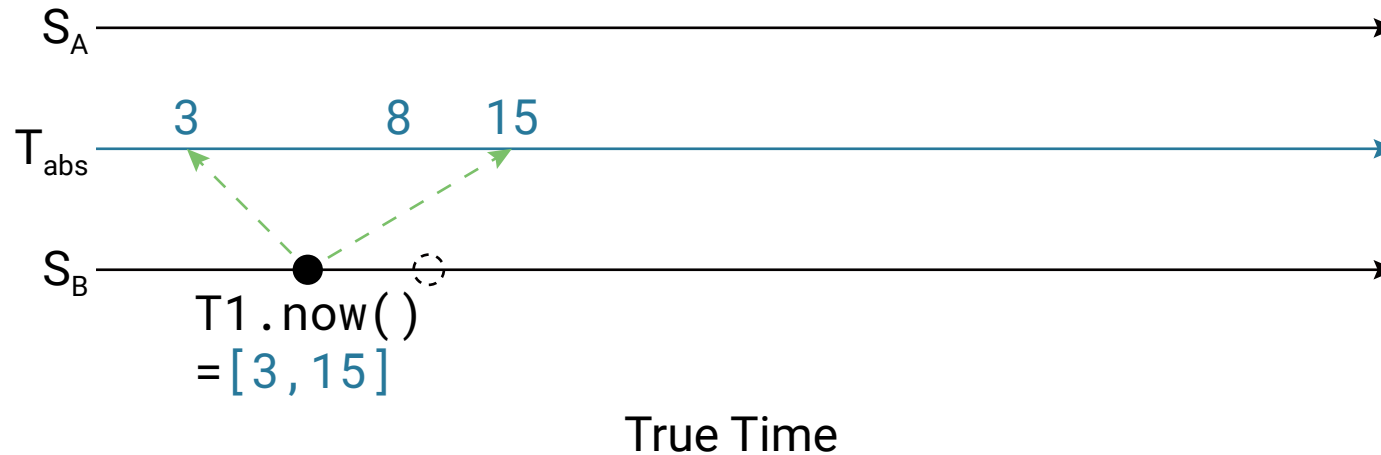
A Brain Teaser Puzzle



- We want: it is always true that $b < d$, how?
 - 1 ($x < y$) and 2 ($c \leq y \leq d$) $\rightarrow x < d$
 - We need to ensure $b < x$; then $b < x < d$, done

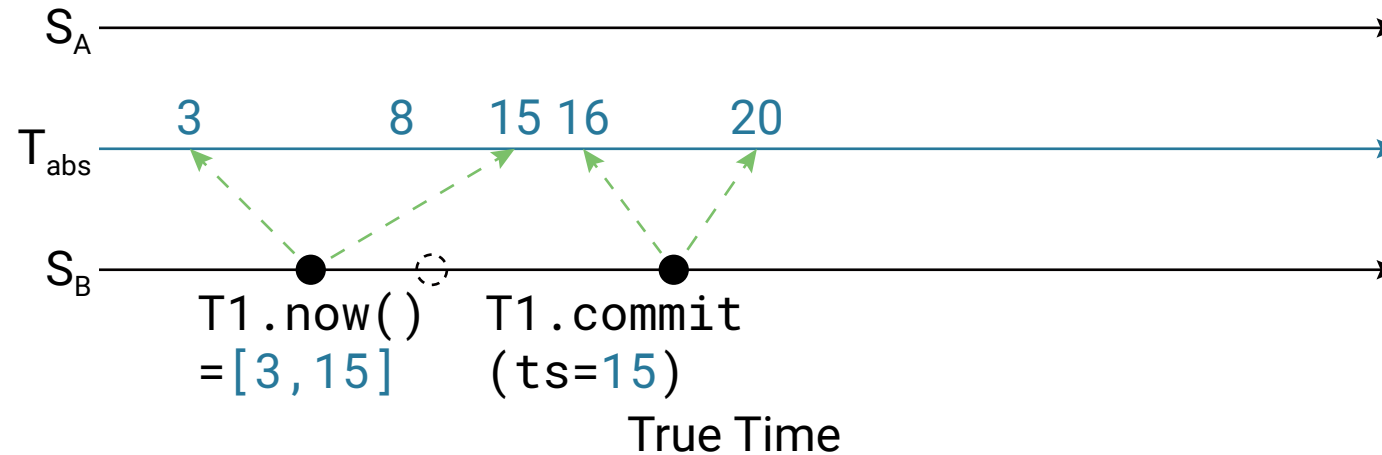
Enforcing the Invariant

- If T2 starts after T1 commits, then T2 must have larger timestamp
- Let T1 write S_B and T2 write S_A



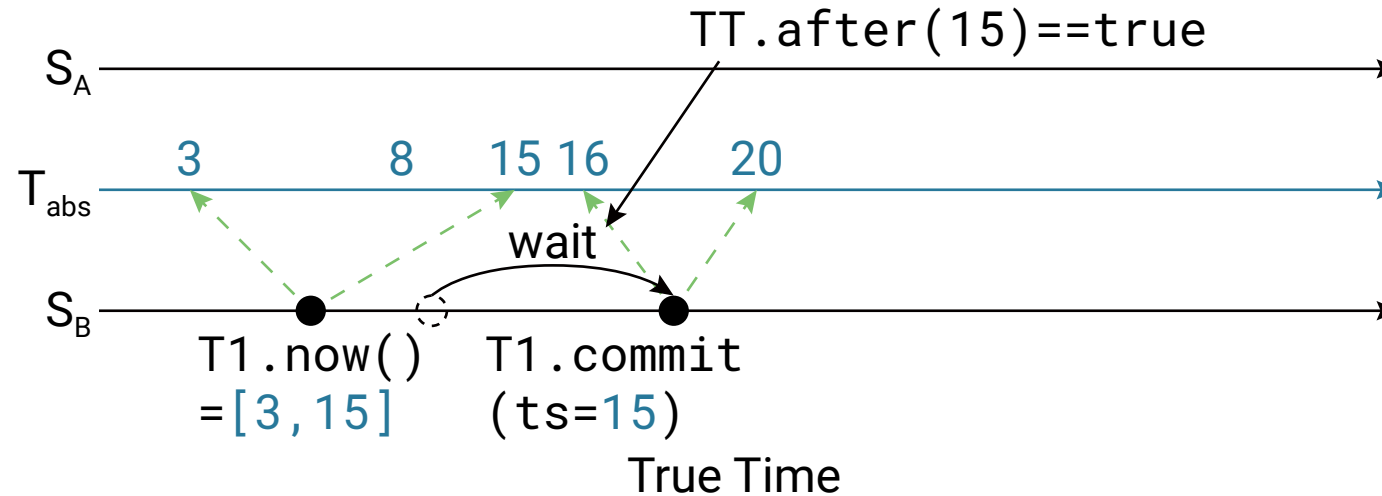
Enforcing the Invariant

- If T2 starts after T1 commits, then T2 must have larger timestamp
- Let T1 write S_B and T2 write S_A



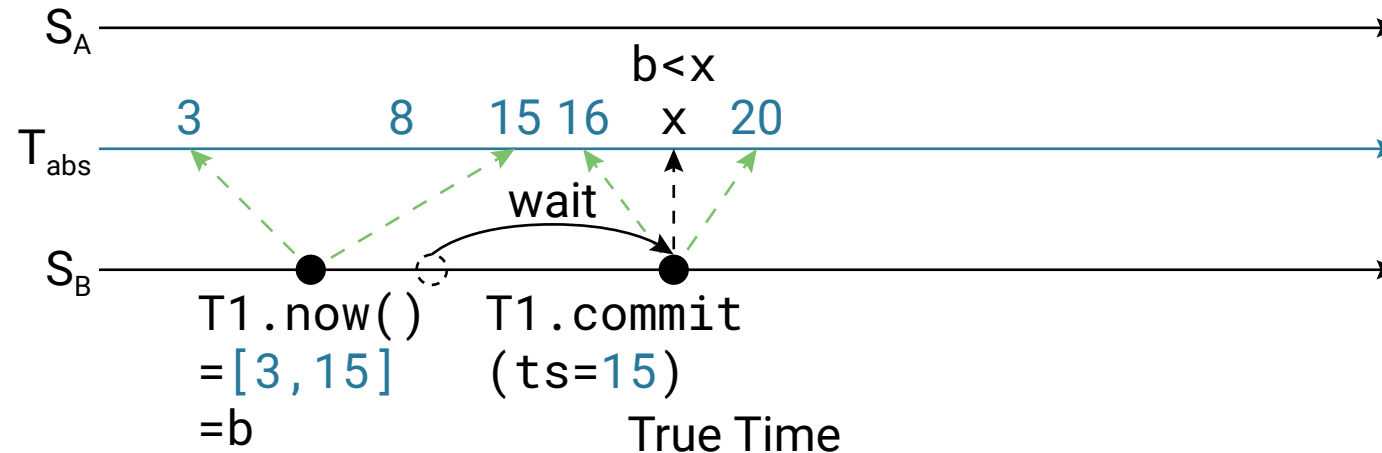
Enforcing the Invariant

- If T2 starts after T1 commits, then T2 must have larger timestamp
- Let T1 write S_B and T2 write S_A



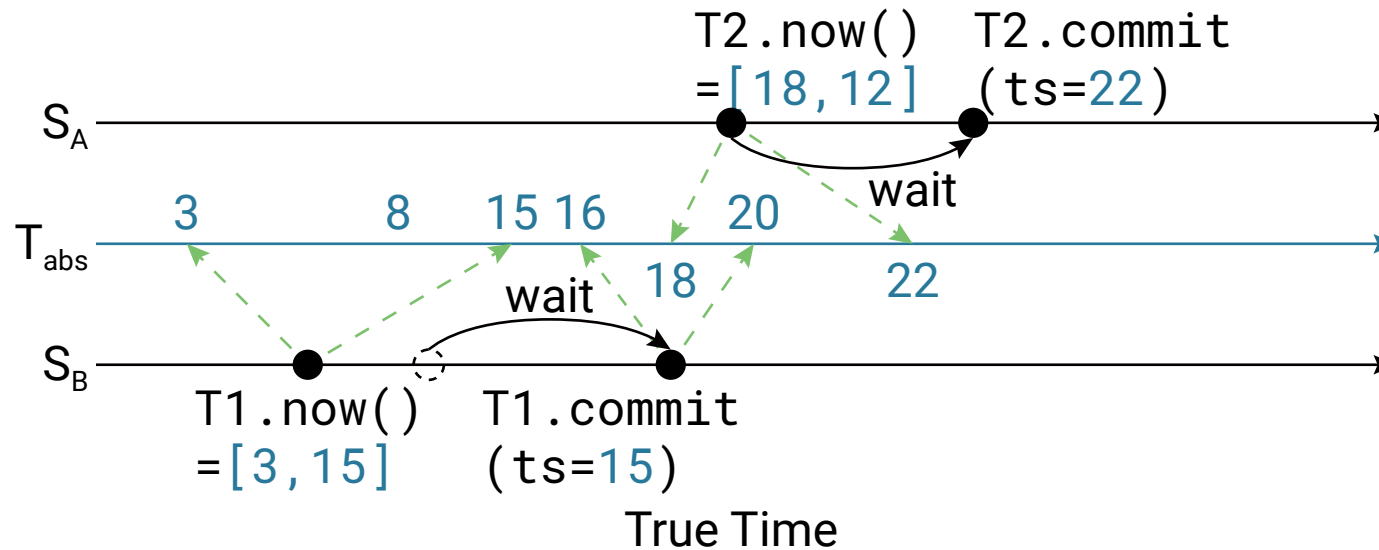
Enforcing the Invariant

- If T2 starts after T1 commits, then T2 must have larger timestamp
- Let T1 write S_B and T2 write S_A



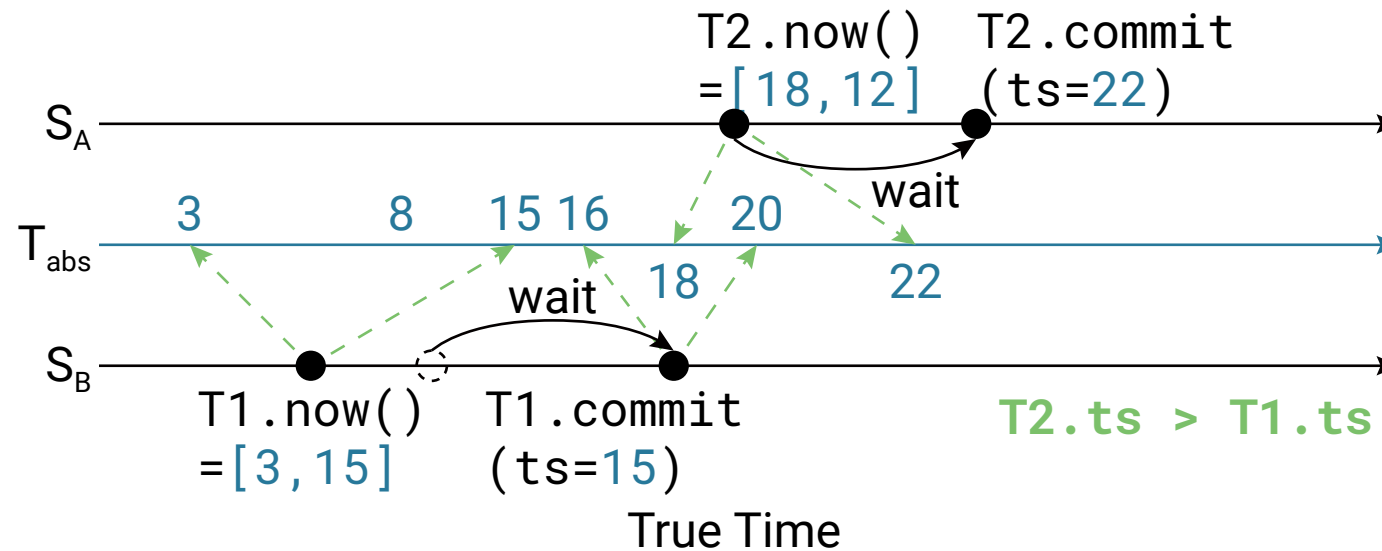
Enforcing the Invariant

- If T2 starts after T1 commits, then T2 must have larger timestamp
- Let T1 write S_B and T2 write S_A



Enforcing the Invariant

- If T2 starts after T1 commits, then T2 must have larger timestamp
- Let T1 write S_B and T2 write S_A



Invariant Takeaways

- The invariant is always enforced: If T2 starts after T1 commits (finishes), then T2 must have a larger timestamp
- How big/small ϵ is does not matter for correctness
- Only need to make sure:
 - `TT.now().latest` is used for ts (in this example)
 - Commit wait, i.e., `TT.after(ts) == true`
- ϵ must be known a priori and small so commit wait is doable!

- How **write** transactions are done
 - 2PL + 2PC (sometimes 2PL for short)
 - How they are timestamped
- How **read-only** transactions are done
 - How read timestamps are chosen
 - How reads are executed

Read-Write Transactions (2 Phase Locking)

- Three phases
 - Execute: do the work and reach a tentative state
 - Prepare: guarantee all participants are ready + durable
 - Commit: enforce a single consistent outcome

Execute → Prepare → Commit



Read-Write Transactions (2 Phase Locking)



- Three phases
 - Execute: do the work and reach a tentative state
 - Prepare: guarantee all participants are ready + durable
 - Commit: enforce a single consistent outcome



Execute → Prepare → Commit

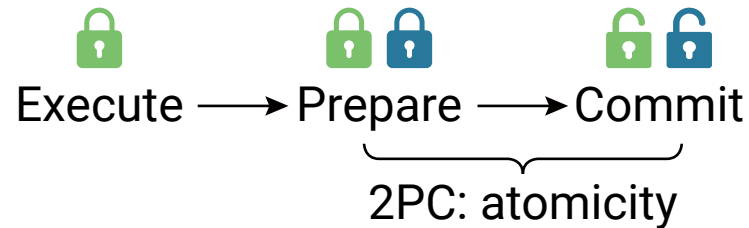
Read-Write Transactions (2 Phase Locking)

- Three phases
 - Execute: do the work and reach a tentative state
 - Prepare: guarantee all participants are ready + durable
 - Commit: enforce a single consistent outcome



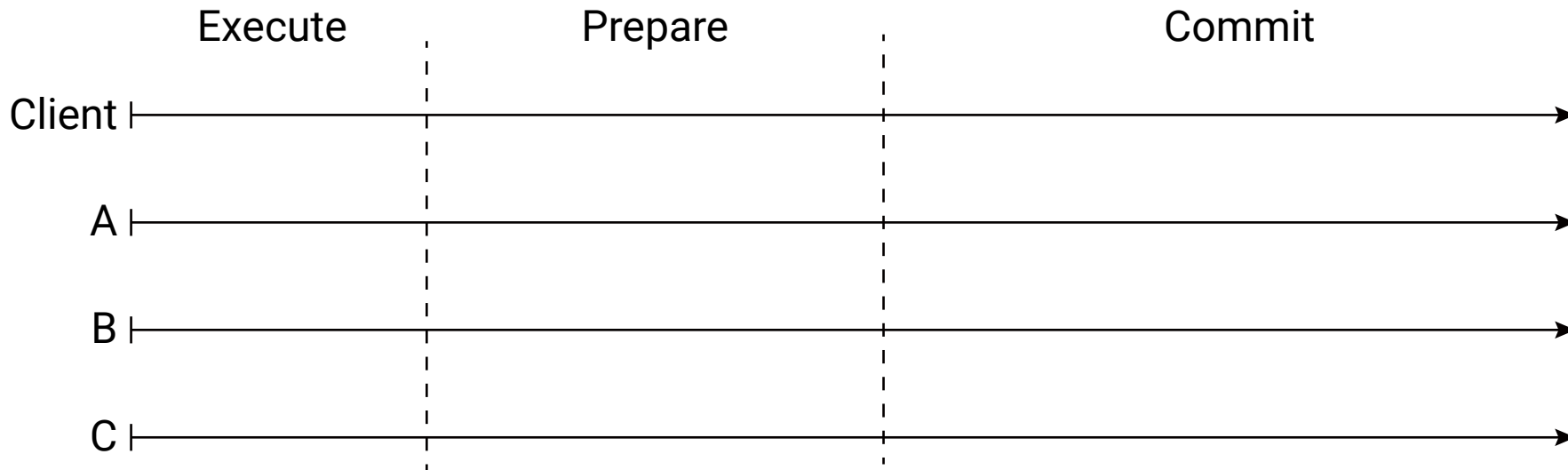
Read-Write Transactions (2 Phase Locking)

- Three phases
 - Execute: do the work and reach a tentative state
 - Prepare: guarantee all participants are ready + durable
 - Commit: enforce a single consistent outcome





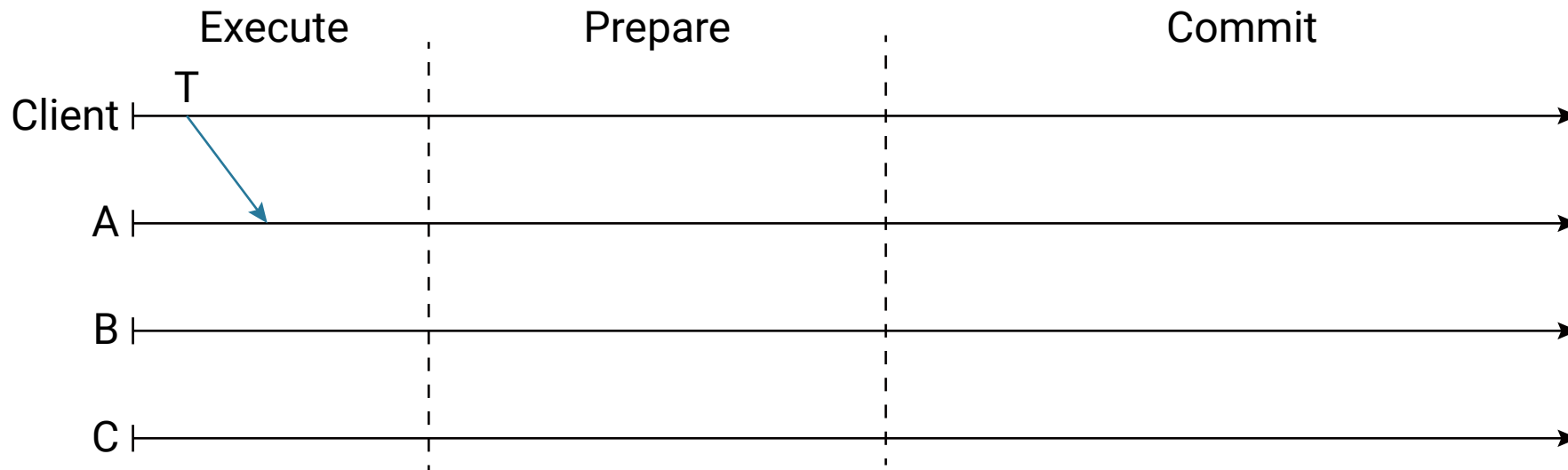
Read-Write Transactions (2 Phase Locking)



- Transaction $T = \{R(A=?), W(A=?+1), W(B=?+1), W(C=?+1)\}$



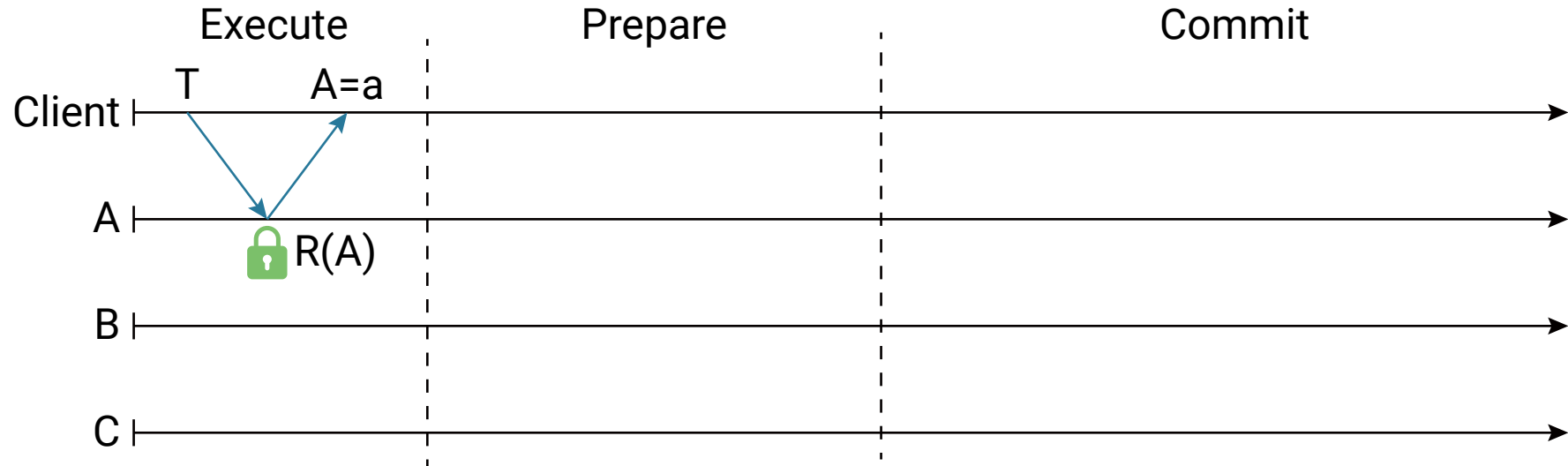
Read-Write Transactions (2 Phase Locking)



- Transaction $T = \{R(A=?), W(A=?+1), W(B=?+1), W(C=?+1)\}$
- **Execute:**
 - Does reads: grab read locks and return the most recent data



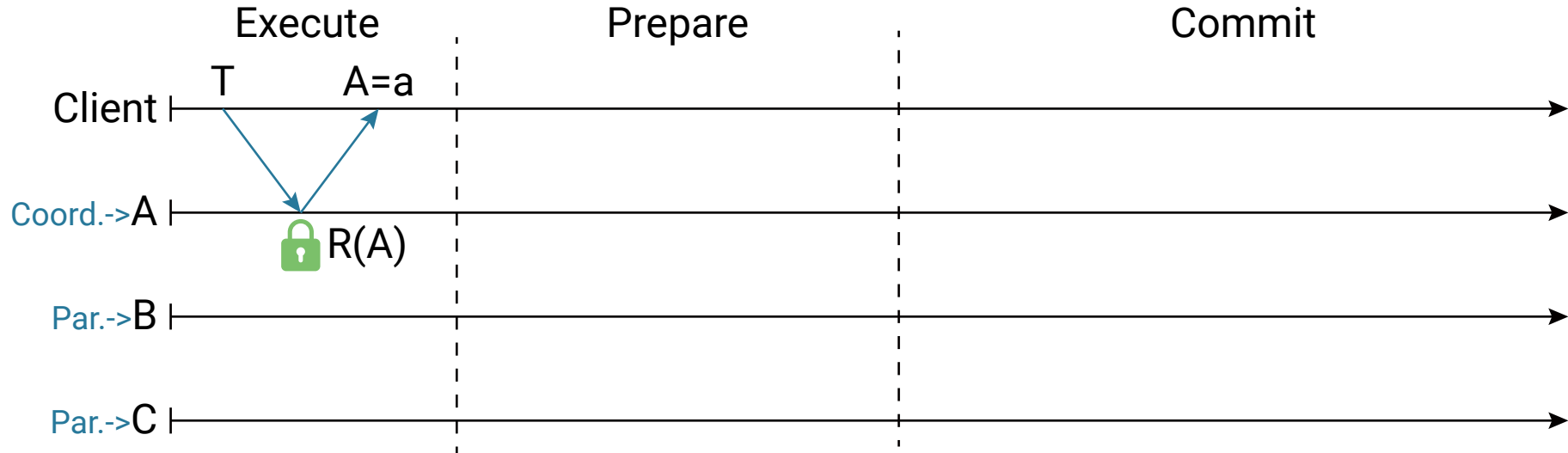
Read-Write Transactions (2 Phase Locking)



- Transaction $T = \{R(A=?), W(A=?+1), W(B=?+1), W(C=?+1)\}$
- **Execute:**
 - Does reads: grab read locks and return the most recent data
 - Client computes and buffers writes locally

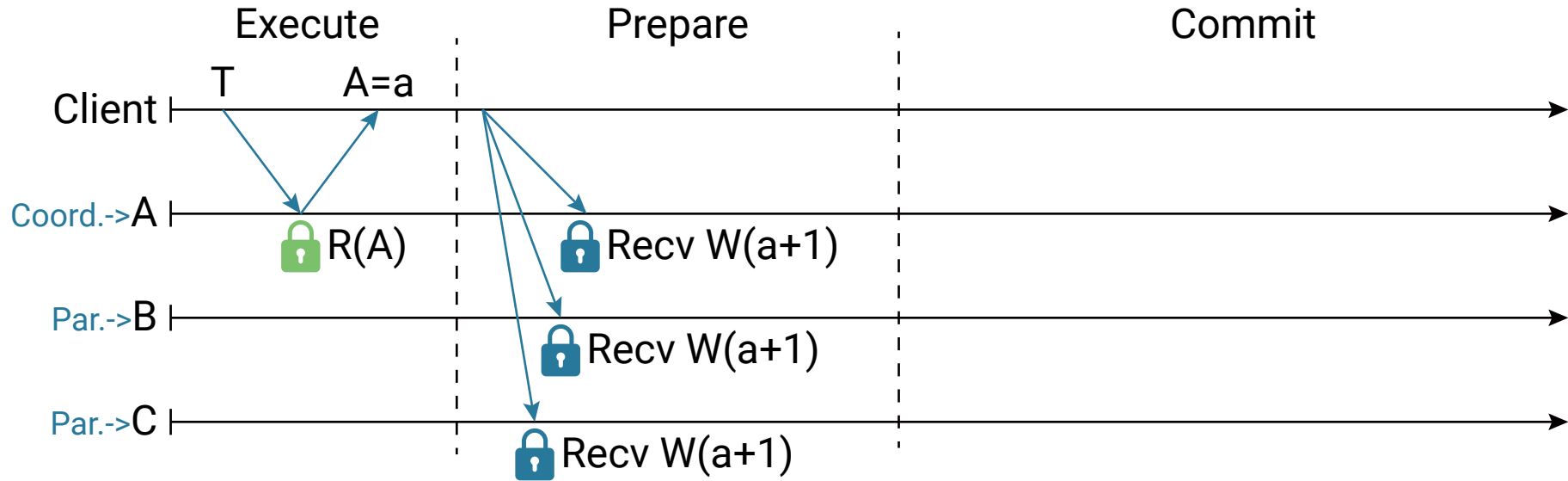


Read-Write Transactions (2 Phase Locking)



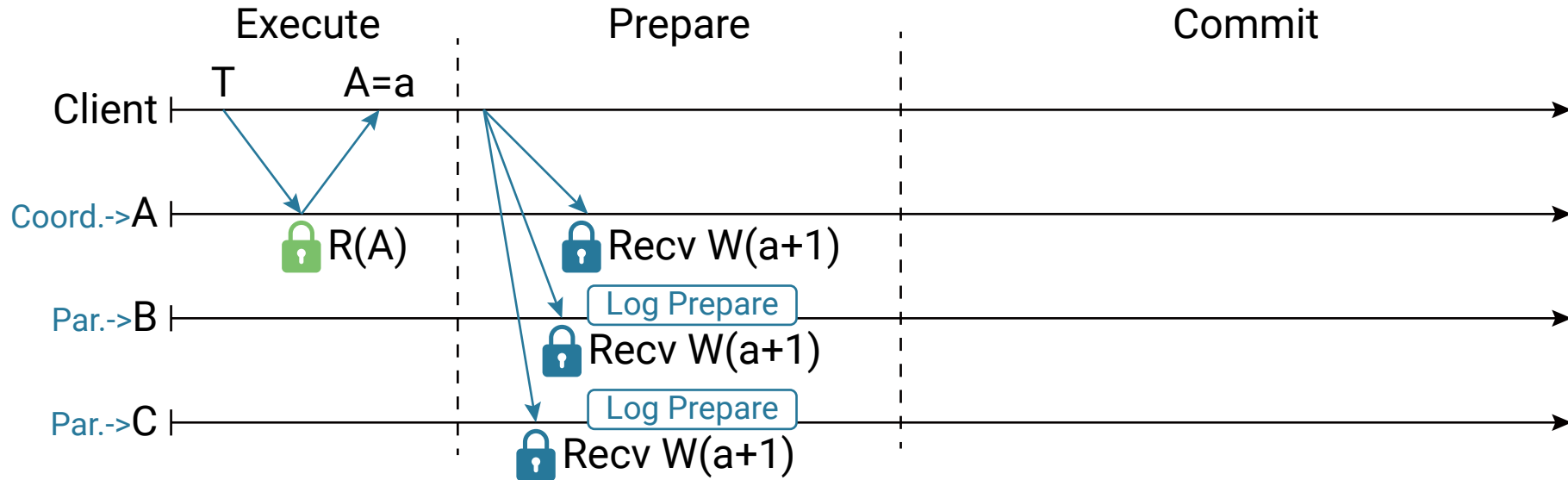
- **Prepare:**
 - Choose a coordinator, e.g., A, others are participants

Read-Write Transactions (2 Phase Locking)



- **Prepare:**
 - ▶ Choose a coordinator, e.g., A, others are participants
 - ▶ Send buffered writes and the identity of the coordinator; grab write locks

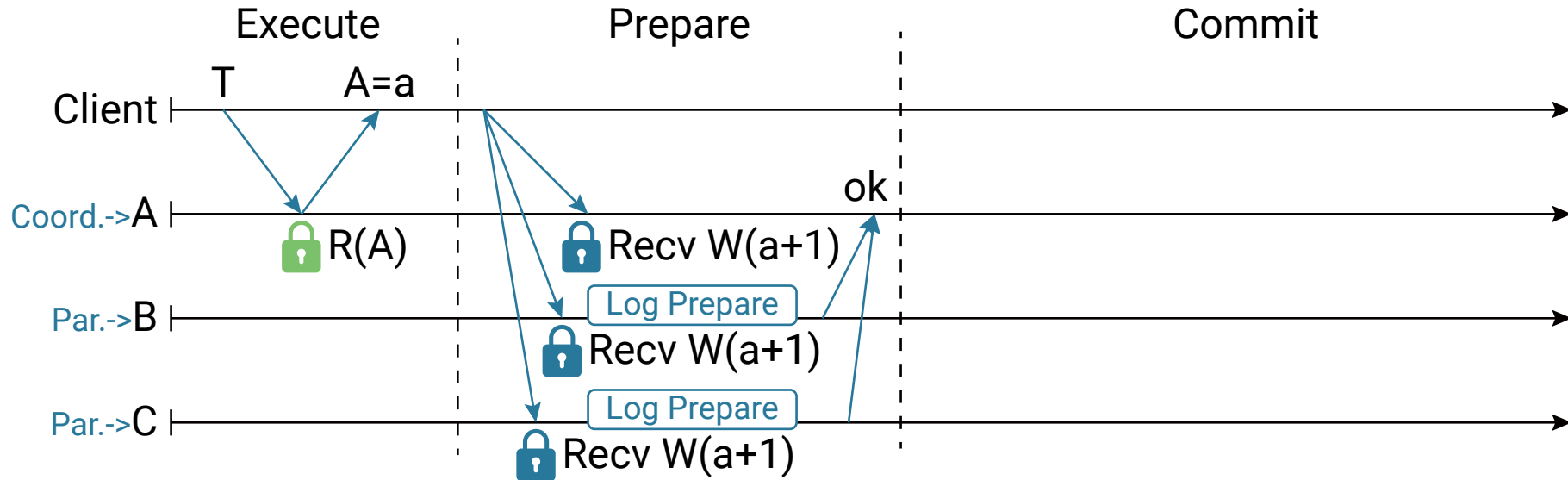
Read-Write Transactions (2 Phase Locking)



- **Prepare:**

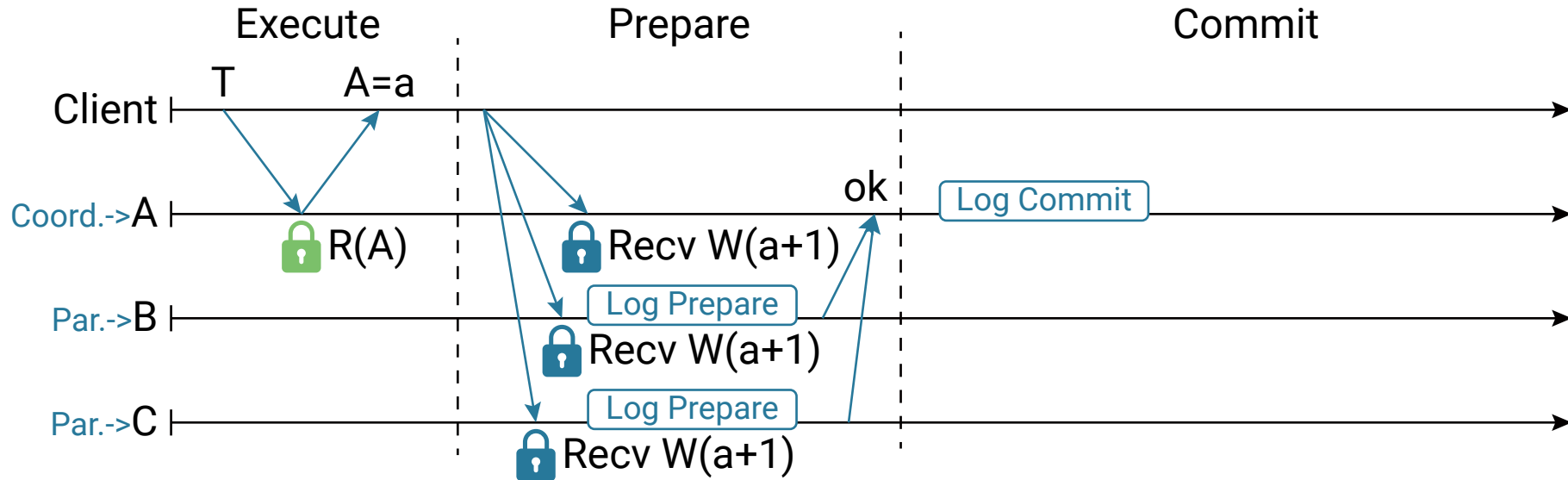
- Each participant prepares T by logging a prepare record via Paxos with its replicas. Coord skips prepare (Paxos Logging)

Read-Write Transactions (2 Phase Locking)



- **Prepare:**
 - Participants send OK to coord if lock grabbed and after Paxos logging is done

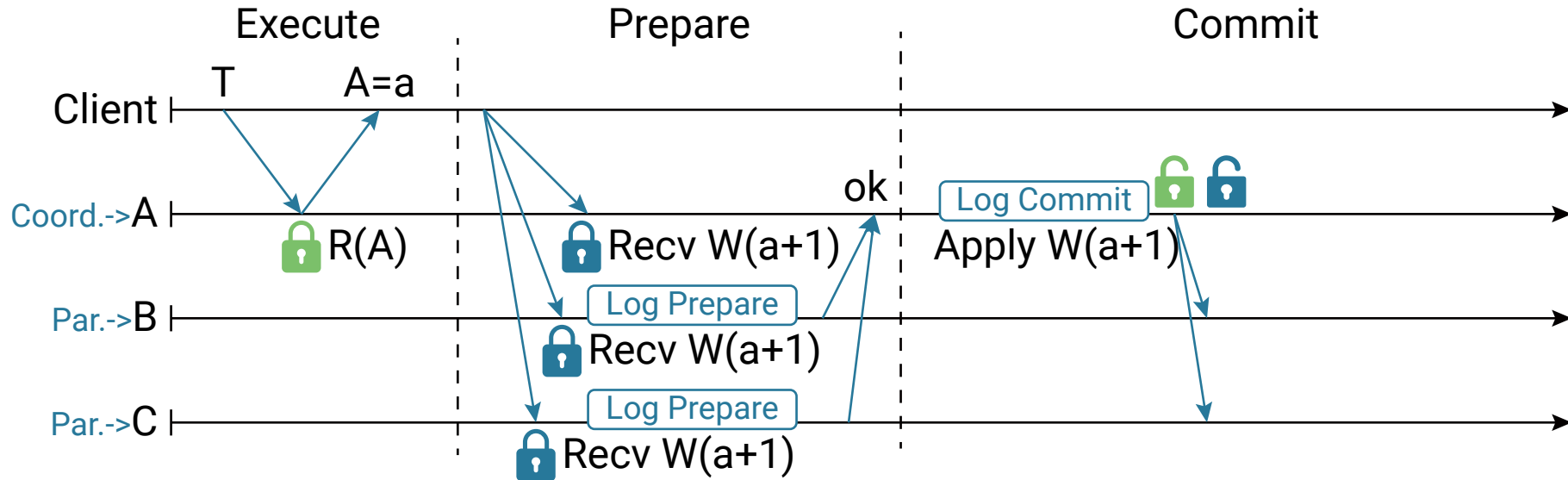
Read-Write Transactions (2 Phase Locking)



- **Commit:**

- ▶ After hearing from all participants, coord commits T if all OK; otherwise, abort T

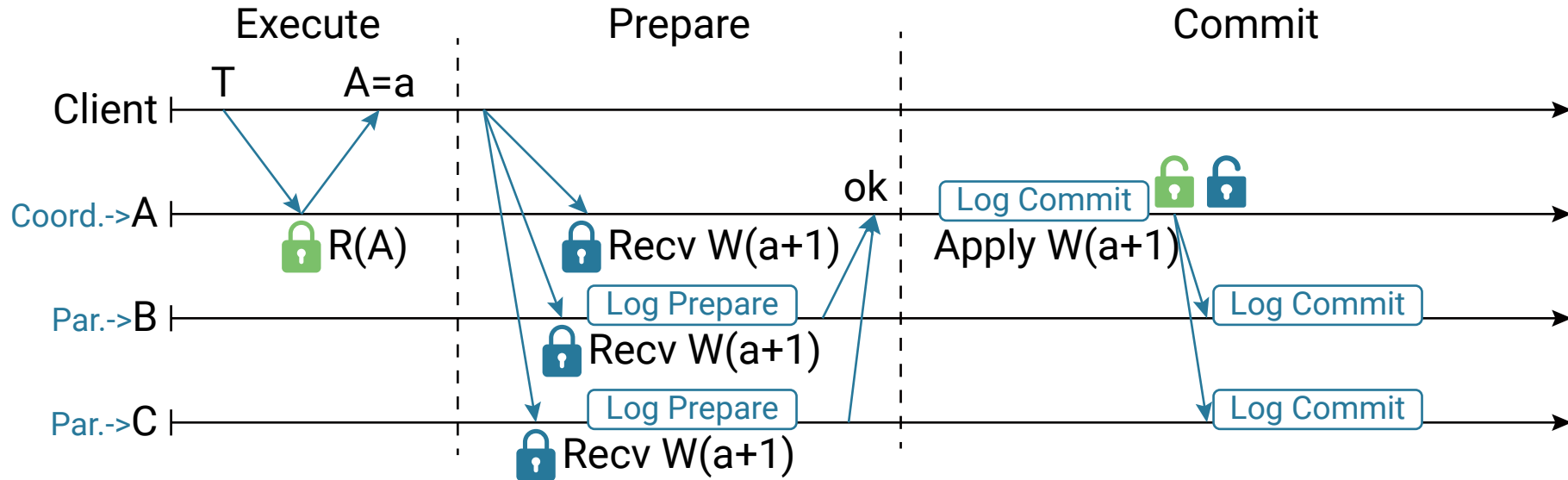
Read-Write Transactions (2 Phase Locking)



- **Commit:**

- Coord logs commit/abort record via Paxos, applies writes if commit, release locks
- Coord sends commit/abort messages to participants

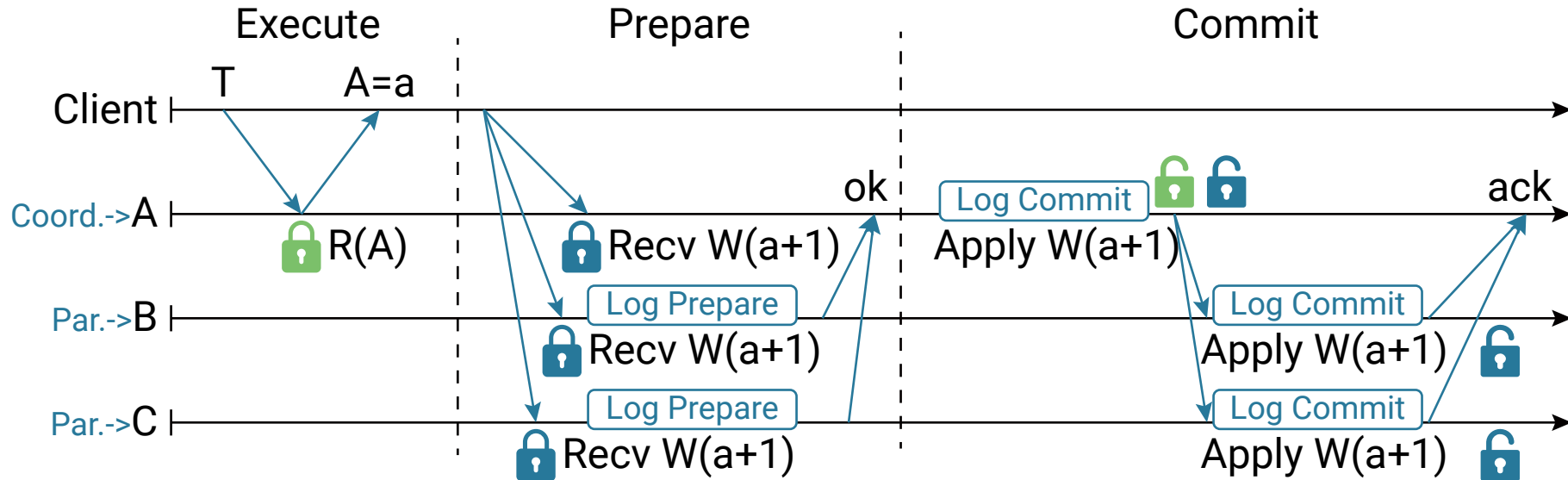
Read-Write Transactions (2 Phase Locking)



- **Commit:**

- Participants log commit/abort via Paxos, apply writes if commit, release locks

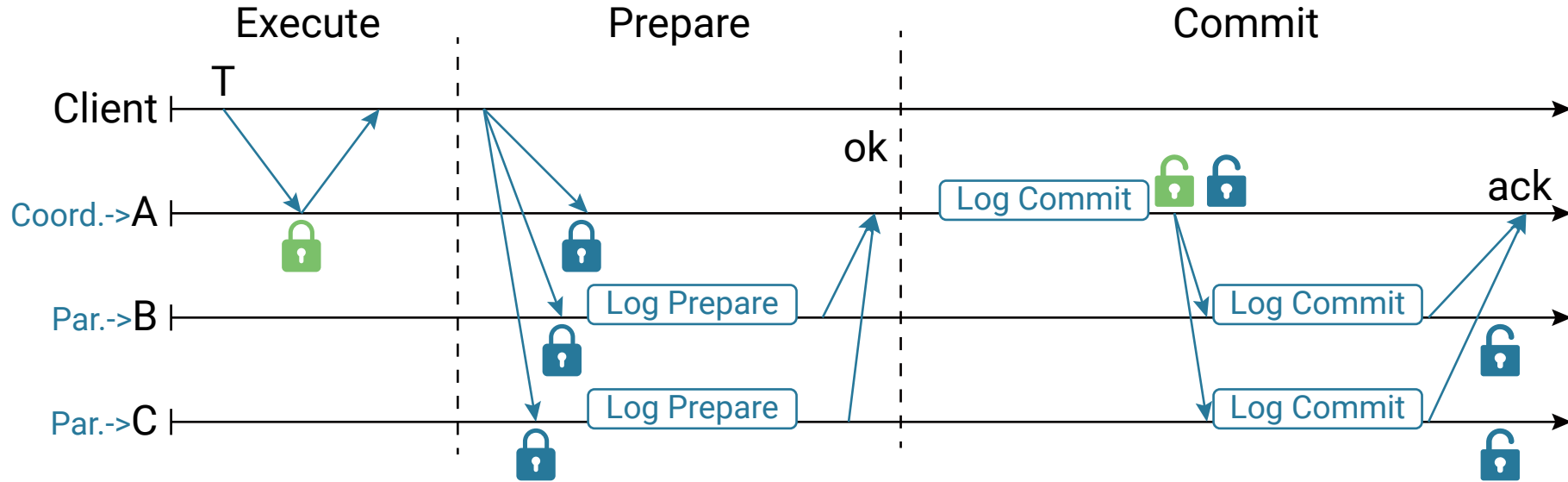
Read-Write Transactions (2 Phase Locking)



- **Commit:**

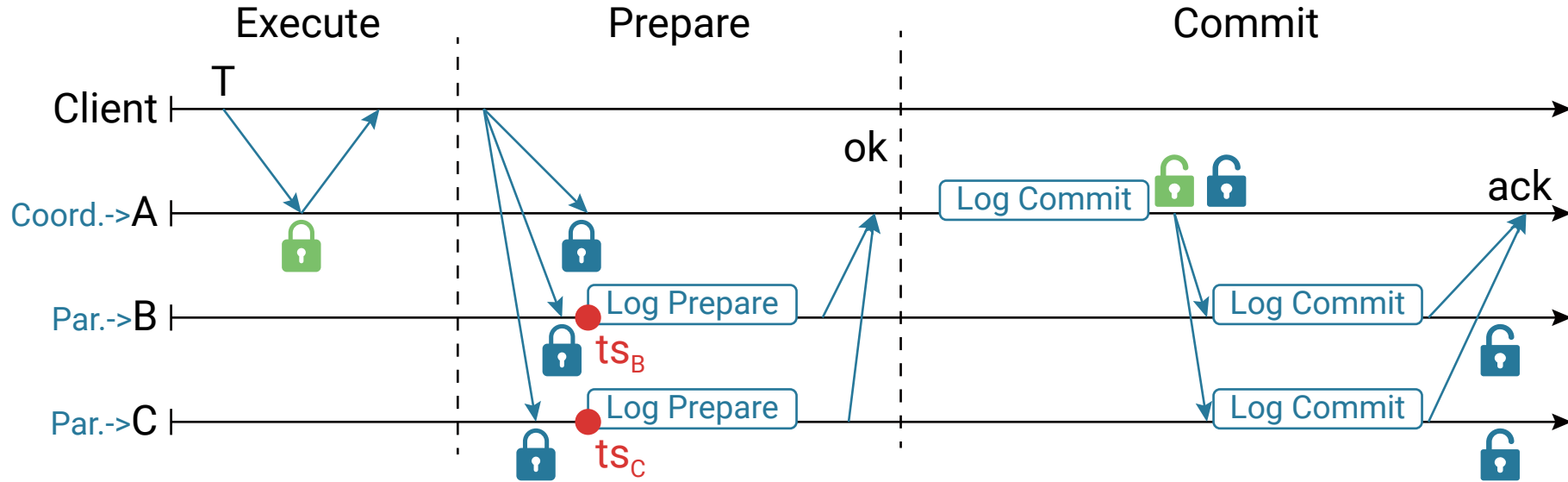
- ▶ Participants log commit/abort via Paxos, apply writes if commit, release locks
- ▶ Coord sends result to client either after its “log commit” or after ack

Timestamping Read-Write Transactions



- **Timestamping:**

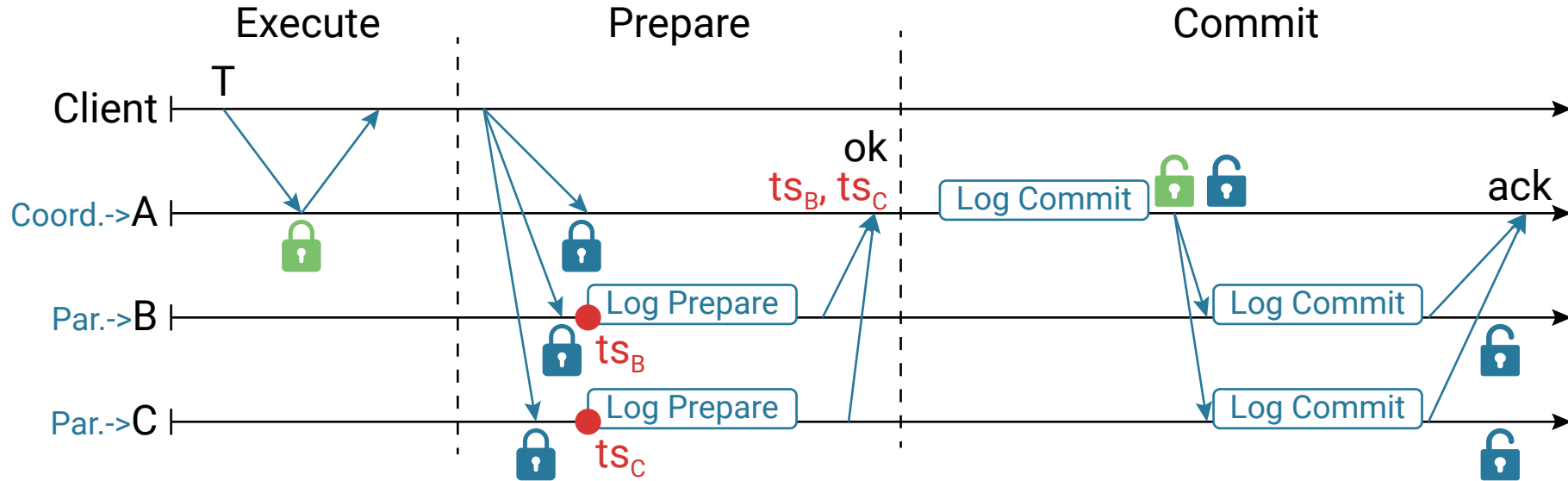
Timestamping Read-Write Transactions



- **Timestamping:**

- Participant: choose timestamp (eg, ts_B and ts_C) larger than any writes it has applied

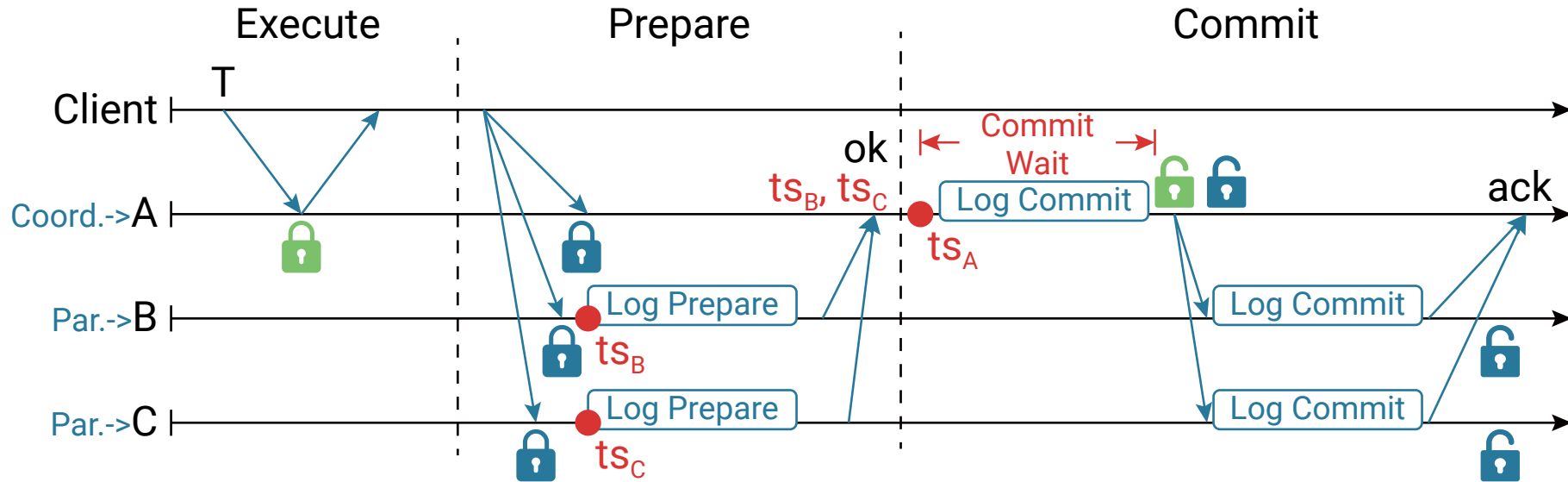
Timestamping Read-Write Transactions



- **Timestamping:**

- ▶ Coordinator: choose a timestamp, e.g., ts_A , larger than
 - Any timestamps proposed by the participants, e.g., ts_B and ts_C
 - Any writes it has applied, and its current $TT.now().latest$

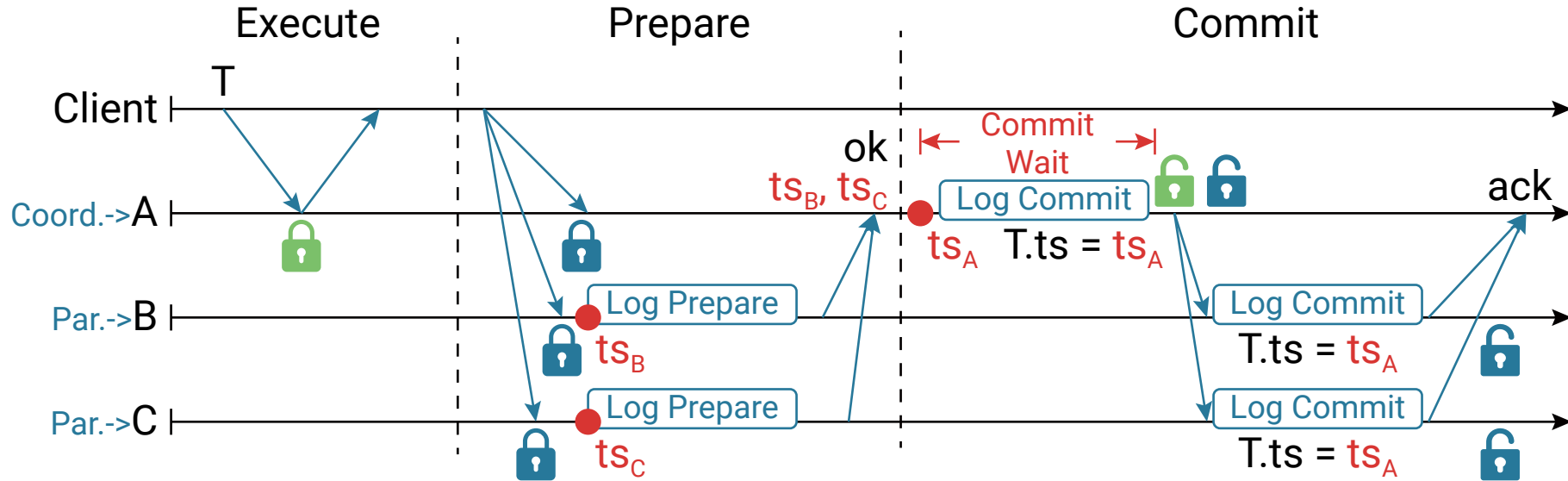
Timestamping Read-Write Transactions



- **Timestamping:**

- Coordinator commit-waits: $TT.after(tsA) == true$. Commit-wait overlaps with Paxos logging

Timestamping Read-Write Transactions

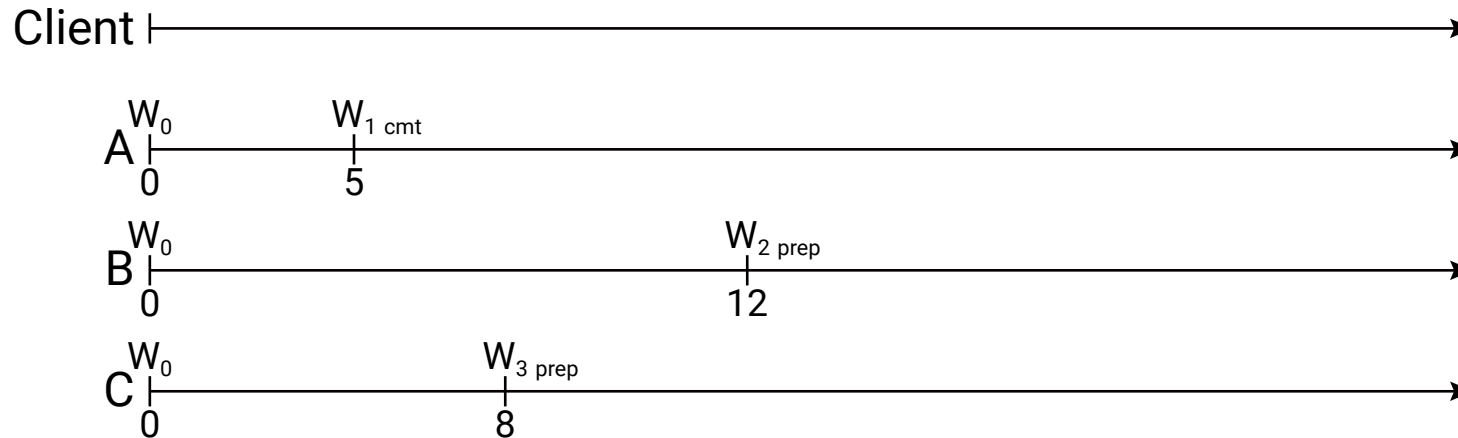


- **Timestamping:**

- ▶ Coordinator commit-waits: $TT.after(ts_A) == true$. Commit-wait overlaps with Paxos logging
 - ts_A is T 's commit timestamp



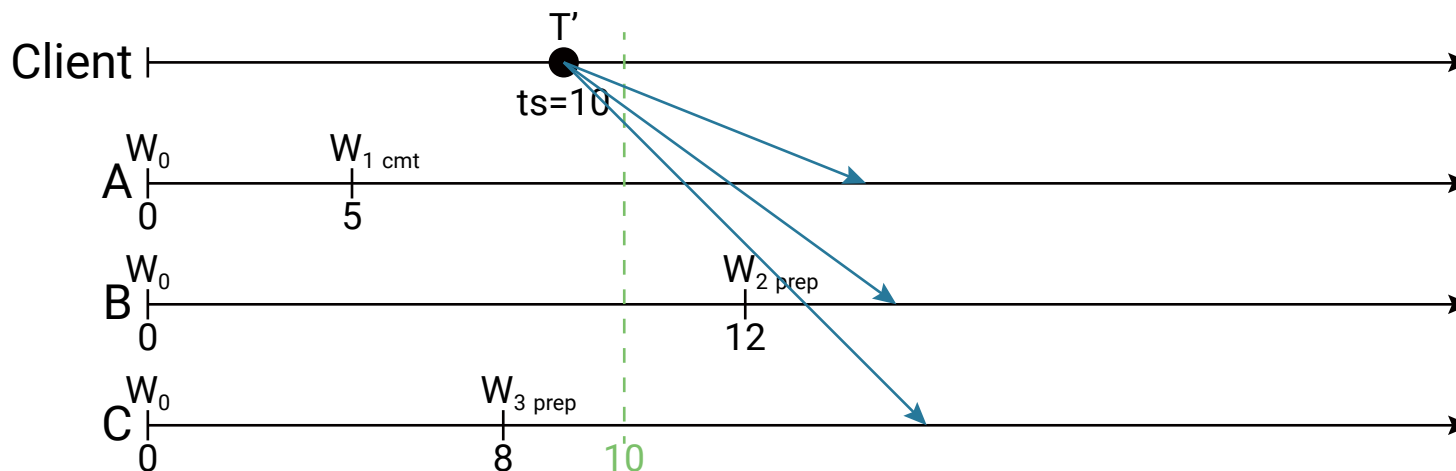
Read-Only Transactions (Shards Part)



- Transaction $T = \{R(A=?, B=?, C=?)\}$

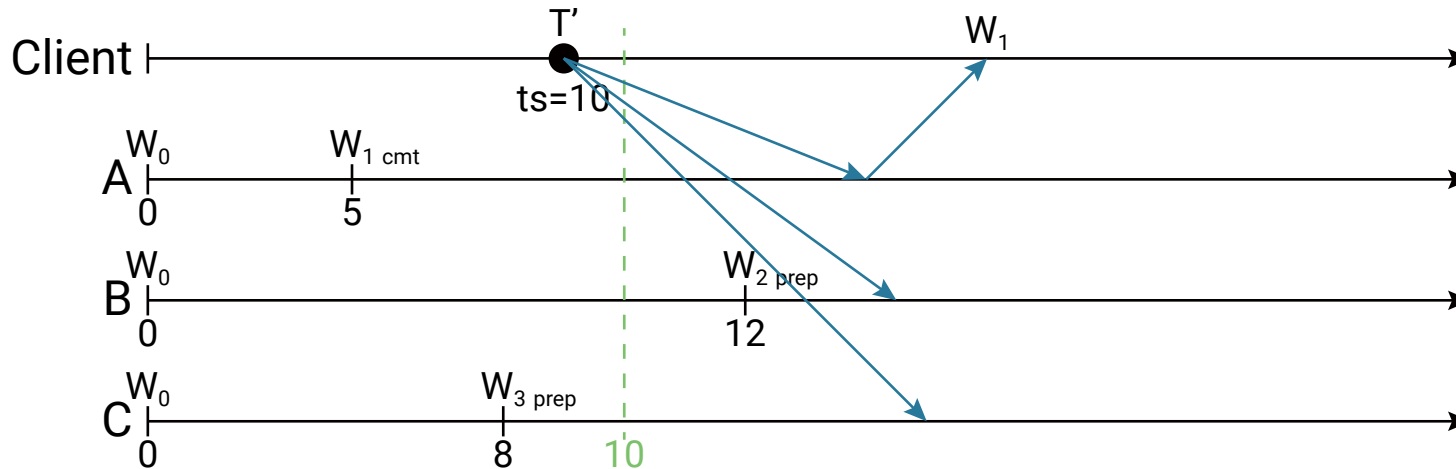


Read-Only Transactions (Shards Part)



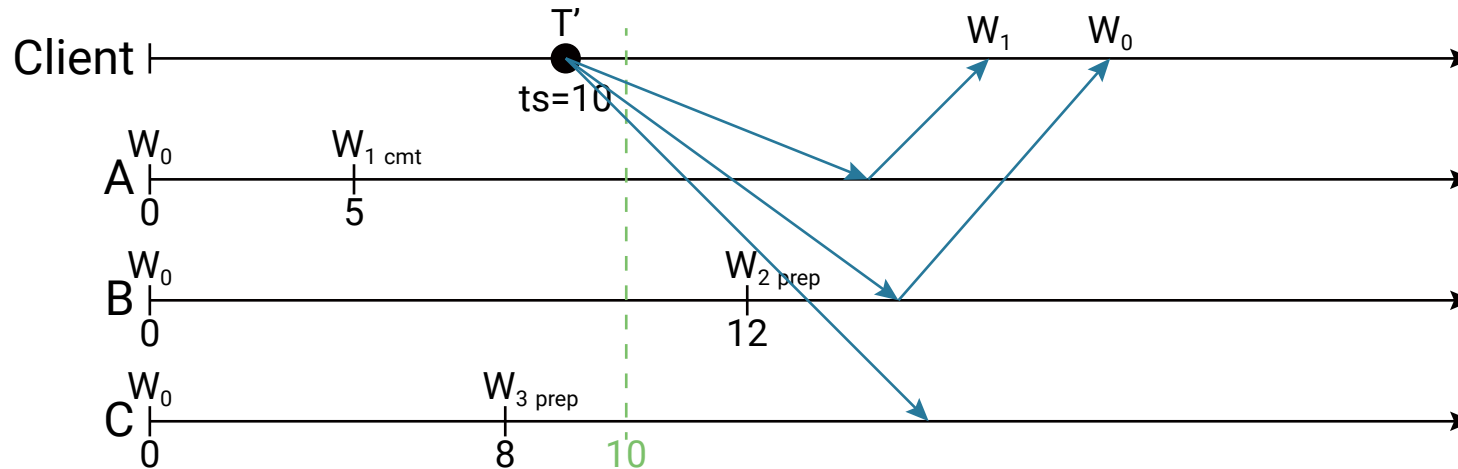
- Transaction $T = \{R(A=?, B=?, C=?)\}$
- Client chooses a read timestamp $ts = TT.now().latest$

Read-Only Transactions (Shards Part)



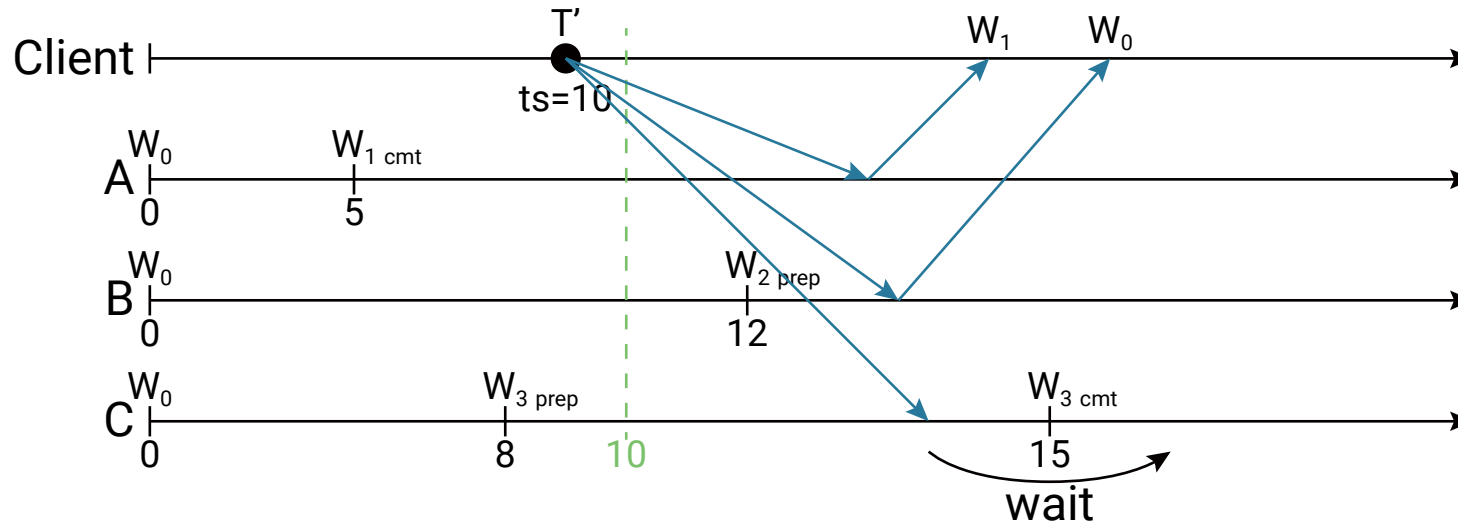
- Transaction $T = \{R(A=?, B=?, C=?)\}$
- Client chooses a read timestamp $ts = TT.now().latest$
- If no prepared write, return the preceding write, e.g., on A

Read-Only Transactions (Shards Part)



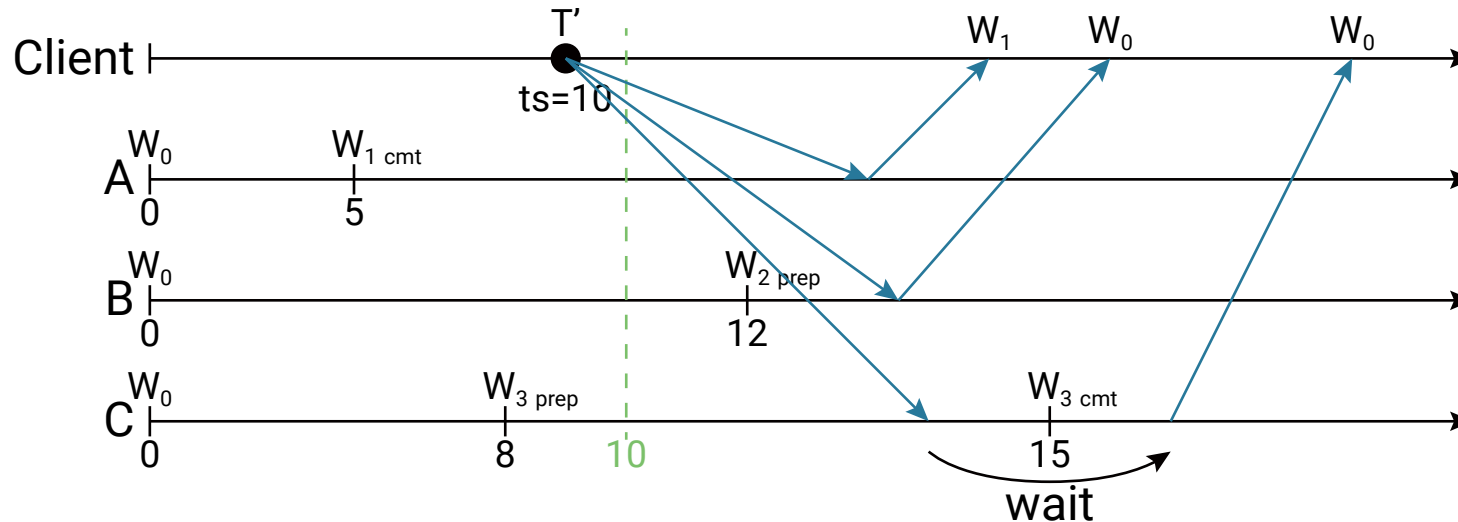
- Transaction $T = \{R(A=?, B=?, C=?)\}$
- Client chooses a read timestamp $ts = TT.now().latest$
- If write prepared with $ts' > ts$, no need to wait, proceed with read, eg, on B

Read-Only Transactions (Shards Part)



- Transaction $T = \{R(A=?, B=?, C=?)\}$
- Client chooses a read timestamp $ts = TT.now().latest$
- If write prepared with $ts' < ts$, wait until write commits, e.g., on C

Read-Only Transactions (Shards Part)



- Transaction $T = \{R(A=?, B=?, C=?)\}$
- Client chooses a read timestamp $ts = TT.now().latest$
- If write prepared with $ts' < ts$, wait until write commits, e.g., on C

Serializable Snapshot Reads

- Client specifies a read timestamp way in the past
 - E.g., one hour ago
- Read shards at the stale timestamp
- Serializable
 - Old timestamp cannot ensure real-time order
- Better performance
 - No waiting in any cases
 - E.g., non-blocking, not just lock-free

Spanner Takeaway

- Strictly serializable (externally consistent)
 - Make it easy for developers to build apps!
- Reads dominant, make them efficient
 - One-round, lock-free
- TrueTime exposes clock uncertainty
 - Commit wait and at least `TT.now.latest()` for timestamps ensure real-time ordering
- Globally-distributed database
 - 2PL w/ 2PC over Paxos!

© 2026 James Jianqiao Yu
Harbin Institute of Technology, Shenzhen

All rights reserved.

These lecture materials are intended solely for educational use in the course *Distributed Systems*. Permission is granted to students enrolled in the course to reproduce these materials, in whole or in part, for personal, non-commercial use only (including printing and photocopying). Any further reproduction or distribution, including but not limited to public posting on websites, social media, forums, or file-sharing platforms, distribution to non-enrolled individuals, or any commercial use, is prohibited without the prior written permission of the author.

This slide deck is adapted from Princeton University's *Distributed Systems* course materials, originally created by Michael Freedman, Kyle Jamieson, and Wyatt Lloyd, and licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 3.0 Unported (CC BY-NC-SA 3.0) License. The original authors and source are gratefully acknowledged.